DTU [Department]
Department of [...]

Embedding Electricity Price Forecasting Benchmarks
in a Production MLOps Platform

Master's Thesis



Embedding Electricity Price Forecasting Benchmarks
in a Production MLOps Platform

Master's Thesis
June 2026

By
[Author Name]

Copyright: Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.
Cover photo: Vibeke Hempler, 2012
Published by: DTU, Department of [...], [Building / street], 2800 Kgs. Lyngby Denmark
www.dtu.dk
ISSN: [0000-0000] (electronic version)
ISBN: [000-00-0000-000-0] (electronic version)

ISSN: [0000-0000] (printed version)
ISBN: [000-00-0000-000-0] (printed version)

Preface

This thesis was prepared at [Department], the Technical University of Denmark (DTU), in partial fulfilment of the requirements for the degree of Master of Science in Engineering.

[Author Name] - [s000000]

.....

Signature

.....

Date

Abstract

Day-ahead electricity price forecasting (EPF) research has produced accurate benchmark models but evaluates them offline, stopping short of the concerns — scheduled retraining, monitoring, and reproducibility — that arise when a model is operated in production. This thesis bridges that gap by embedding the benchmark EPF models of Lago et al. (the LEAR ensemble and a deep neural network) and the Transformer of Llorente and Portela in an end-to-end MLOps platform, and evaluating both the forecasts and the system that serves them. The platform ingests ENTSO-E data on a schedule, stores it in ClickHouse, recalibrates and forecasts daily for the DK1 and DE bidding zones, monitors data and model health, and records the provenance of every run. The three models are replicated to specification and compared on a shared recent test window with paired Diebold–Mariano and Giacomini–White significance tests: LEAR and a tuned DNN ensemble are statistically tied as the most accurate models on both markets ($p > 0.16$), both significantly outperform the pure Transformer ($p \leq 0.012$), and all three beat the weekly-naive baseline; the source papers' model rankings do not replicate under our reduced tuning budget. The system is assessed against the nine MLOps practices of Zarour et al.: eight are realised end-to-end and one is minimally realised, supported by an ML Test Score of 2.5 (bound by data testing), 66.7 % provenance-triple coverage backed by a re-run that reproduced a logged forecast to within 7×10^{-6} EUR/MWh, and DORA throughput of 20 deployments at a sub-minute lead time. The outcome is a reproducible, monitored EPF platform together with a faithful, significance-tested model comparison on current market data.

Acknowledgements

[Acknowledgements to be added.]

Contents

Preface . . . ii
Abstract . . . iii
Acknowledgements . . . iv
Acronyms . . . ix

1 Introduction 1
1.1 Motivation . . . 1
1.2 The gap between forecasting research and operation . . . 1
1.3 Aim and research questions . . . 1
1.4 Contributions . . . 2
1.5 Report structure . . . 2

2 Background and Related Work 3
2.1 Day-ahead electricity price forecasting . . . 3
2.2 The Lago et al. open-access benchmark . . . 3
2.3 Transformer approaches to EPF . . . 4
2.4 MLOps: practices and maturity . . . 4
2.5 Evaluating machine-learning systems . . . 4

3 Methods — Forecasting Models and Evaluation 5
3.1 Replication framing . . . 5
3.2 Data and features . . . 5
3.3 Models . . . 6
3.4 Recalibration and backtesting . . . 8
3.5 Evaluation protocol . . . 8
3.6 Hyperparameter tuning . . . 9
3.7 Deviations from the source papers . . . 9

4 System Implementation 10
4.1 Scope and approach . . . 10
4.2 System at a Glance . . . 10
4.3 Zarour's Nine MLOps Practices — Implementation . . . 11
4.4 Cross-cutting: the reproducibility–monitoring link and MVP scope . . . 18
4.5 Deferred tooling and adoption triggers . . . 19

5 Evaluation Framework 23
5.1 Approach . . . 23
5.2 Test inventory . . . 23
5.3 Why these five tests . . . 24
5.4 ML Test Score . . . 24
5.5 Reproducibility tests . . . 25
5.6 Delivery throughput (DORA) . . . 25
5.7 Drift monitoring counts . . . 25
5.8 Maturity-model positioning . . . 26
5.9 Threats to validity (preview) . . . 26
5.10 What is not tested and why . . . 27

6 Results — Forecasting Models **28**
6.1 Headline accuracy . . . 28
6.2 Statistical significance . . . 29
6.3 Computation cost . . . 29
6.4 Comparison with published benchmarks . . . 30
6.5 Threats to validity (modelling side) . . . 31
6.6 Future work (modelling side) . . . 31

7 Results — System Evaluation **32**
7.1 ML Test Score . . . 32
7.2 Reproducibility . . . 32
7.3 Delivery throughput . . . 33
7.4 Drift detection as monitoring, not a retraining trigger . . . 34
7.5 Maturity placement . . . 34
7.6 Threats to validity . . . 34
7.7 Known gaps and future work . . . 34
7.8 Summary . . . 35

8 Discussion **37**
8.1 RQ1 — Comparing the forecasting models . . . 37
8.2 RQ2 — Engineering trade-offs of productionising an EPF benchmark . . . 38
8.3 RQ3 — Critical practices and the limits of a single-host platform . . . 38
8.4 Tying the threads together . . . 38
8.5 Limitations . . . 39

9 Conclusion and Future Work **40**
9.1 Future work . . . 40

Bibliography **43**

A Glossary **45**

B ML Test Score Worksheet **46**

List of Figures

4.1 Host-level architecture. A single Docker Compose host runs ClickHouse, the Prefect server and worker, Loki, and Grafana. Flow runs spawn ephemeral `algo-repo:latest` containers that read ClickHouse, call ENTSO-E, and emit logs back to Loki. 11
4.2 Data plane. ENTSO-E data flows through typed mappers into ClickHouse, where the forecasting pipeline reads it, trains and evaluates models, and writes predictions and artifacts back. Reconciliation and drift detection close the loop on a daily cadence. 12
4.3 CI/CD pipeline. GitHub Actions runs the PR gate and the deploy gate; the deploy gate reaches the production host across the Tailscale link, rebuilds the image, redeploys Prefect, and triggers a smoke forecast. 14
4.4 Drift-triggered retraining loop. Drift events land in `algo.drift_reports` and post to Slack; configs with `retrain_on_drift=True` additionally trigger a retraining flow that records each event in `algo.retrain_events`. 19

List of Tables

3.1 Datasets. Both markets span 2022-03-10 to 2026-05-31 (ENTSO-E Transparency Platform). . . . 5
3.2 Model setups (one row per model; identical across both markets unless noted). . . . 6
3.3 Tuned DNN hyperparameters per market. Selected by the protocol of §3.6 on the ex-ante validation window; the architecture-only search is re-run whenever the feature set changes, so these values are refreshed by the paper-exogenous re-tune that accompanies the final benchmark. . . . 7
3.4 Documented deviations from [3] and [6]. . . . 9

4.1 Operational inventory of Prefect deployments. Cron strings are UTC. . . . 21
4.2 Technology stack and rationale. . . . 22

5.1 Two kinds of test used in this work. . . . 23
5.2 Tests applied to the system in this work. . . . 23
5.3 Mapping of tests to the nine MLOps practices in Zarour et al. [4]. . . . 24
5.4 Scoring rule applied to each of the 28 items. . . . 24
5.5 The four sections of the rubric and what each is designed to detect. . . . 25
5.6 Two reproducibility tests. . . . 25
5.7 DORA: what is measured, what is excluded, and why. . . . 26
5.8 The five MLOps maturity models reviewed by Zarour et al. [4], Table 7. All derive from CMM/CMMI. . . . 26
5.9 Threats to validity at a glance; each is discussed under §7.6. . . . 27
5.10 Explicit non-claims of this evaluation framework. . . . 27

6.1 DK1: final benchmark metrics (test 2025-10-24–2026-05-31, common days). 28
6.2 DE: final benchmark metrics (test 2025-10-24–2026-05-31, common days). 28
6.3 DK1: GW multivariate p-values (column model better than row model). . . . 29
6.4 DE: GW multivariate p-values (column model better than row model). . . . 29
6.5 Amortised recalibration cost (full-backtest wall-clock / common test day). Full timings in `Report/metrics/run_timings.md`. . . . 29
6.6 Our ensemble rMAE vs published ([3] Table 3). Periods differ; rankings/levels only. . . . 30

7.1 ML Test Score self-assessment (min-of-sections rule). Per-item evidence in Appendix B. . . . 32
7.2 Provenance triple coverage across 15 live runs in the window. . . . 33
7.3 Reproducibility re-run: regenerated from logged provenance, compared against the stored prediction. . . . 33
7.4 Maturity assessment of Zarour's nine practices, with the Google MLOps maturity model level [10] assigned to each. . . . 36

B.1 Section 1 — Data tests (sum 2.5 / 7). . . . 46
B.2 Section 2 — Model development tests (sum 3.0 / 7). . . . 46
B.3 Section 3 — ML infrastructure tests (sum 4.0 / 7). . . . 47
B.4 Section 4 — Monitoring tests (sum 4.0 / 7). . . . 47

Acronyms

CI/CD Continuous Integration / Continuous Delivery

DM Diebold–Mariano (test of equal predictive accuracy)

DNN Deep Neural Network

DORA DevOps Research and Assessment

ENTSO-E European Network of Transmission System Operators for Electricity

EPF Electricity Price Forecasting

GW Giacomini–White (test of conditional predictive ability)

LEAR Lasso Estimated AutoRegressive (model)

MAE Mean Absolute Error

MLOps Machine Learning Operations

rMAE relative Mean Absolute Error

RMSE Root Mean Squared Error

sMAPE symmetric Mean Absolute Percentage Error

TPE Tree-structured Parzen Estimator

VST Variance-Stabilising Transformation

1 Introduction

1.1 Motivation

Day-ahead electricity price forecasting (EPF) is the task of predicting the wholesale price of electricity for each delivery period of the following day in a given bidding zone, from a combination of historical prices and exogenous signals such as load and renewable-generation forecasts. Accurate day-ahead forecasts are essential to generators, retailers, and grid operators, who rely on them to schedule production, hedge positions, and bid into the day-ahead auction. The task has grown harder over the past decade. The expansion of wind and solar generation introduces supply-side volatility that statistical models struggle to capture, and the coupling of European bidding zones propagates price shocks across borders in ways historical data may not reflect. The resulting price series are heavy-tailed and spike-prone and, increasingly under high renewable infeed, admit negative prices — a regime that rules out modelling choices, such as a logarithmic price transform, that presume positivity [1].

1.2 The gap between forecasting research and operation

Research on EPF has produced increasingly accurate models, spanning statistical, reduced-form, and machine-learning approaches, and the field has been surveyed comprehensively [1, 2]. A turning point was the open-access benchmark of Lago et al. [3], which documented how severe the evaluation problem had become — new methods compared on private datasets, over test periods too short to be conclusive, and against baselines too weak to be informative — and responded by standardising both a pair of benchmark models and a methodology for comparing them rigorously.

Yet even the most careful EPF contributions stop at the boundary of offline evaluation: they standardise how models are *compared*, but not how they are *run*. Ingesting data on a schedule, versioning experiments, detecting model degradation, and retraining on fresh data are left unaddressed. This gap between building a model and operating it is precisely the one that Machine Learning Operations (MLOps) emerged to close, by carrying DevOps principles — automation, reproducibility, and monitoring — into machine-learning systems. Zarour et al. [4] synthesise the MLOps literature into a systematic review of nine core practices and an accompanying maturity model. The two bodies of work rarely meet: EPF research treats forecasting as a problem that ends when predictions are produced, while MLOps research is largely domain-agnostic. The closest precedent for joining them is the work of Subramanya et al. [5], who apply an MLOps pipeline to electricity-market forecasting as a case study.

1.3 Aim and research questions

This thesis takes the benchmark EPF models from the forecasting literature and embeds them in an end-to-end production platform built along MLOps principles. The platform ingests data from the ENTSO-E Transparency Platform on a recurring schedule, stores it in ClickHouse, produces day-ahead forecasts through rolling-window recalibration, monitors prediction quality and data drift, and maintains a complete audit trail from every forecast back to the code version and data window that produced it. It is evaluated on two axes: the *forecast quality* of the models it serves, and the *operational maturity* of the system that serves them. Three research questions structure the investigation.

RQ1 (forecasting). How do faithfully-replicated LEAR, DNN, and Transformer day-ahead EPF models compare on recent ENTSO-E data for the DK1 and DE bidding zones — against one another, a weekly-naive baseline, and the rankings reported by Lago et al. [3] and Llorente and Portela [6]?

RQ2 (engineering). What engineering trade-offs and operational constraints emerge when embedding these benchmark models into a scheduled, containerised, continuously-monitored production pipeline?

RQ3 (maturity). Assessed against the nine MLOps practices of Zarour et al. [4] and the Google maturity model, which practices does the platform realise, which prove most critical, and where does a single-host minimum-viable platform reach its limit?

1.4 Contributions

This work makes four contributions.

- **A production EPF platform.** An end-to-end system that ingests live ENTSO-E data, recalibrates and forecasts on a daily schedule, monitors data and model health, and records the full provenance of every run — the artefact that RQ2 and RQ3 are evaluated against.
- **A faithful model replication.** LEAR and the DNN of Lago et al. [3] and the Transformer of Llorente and Portela [6], reimplemented to specification and compared across two markets on a shared recent test window with paired Diebold–Mariano and Giacomini–White significance tests (RQ1).
- **A composed quantitative MLOps evaluation.** A combination of the ML Test Score rubric, provenance-coverage measurement, a worked reproducibility re-run, and DORA delivery-throughput metrics, assembled to give the otherwise-qualitative maturity question a measurable answer.
- **A practice-by-practice maturity assessment** of the platform against Zarour et al.'s nine practices, with each practice traced to a concrete repository artefact and positioned on the Google maturity model.

The scope is deliberately bounded. The work forecasts point prices, not probabilistic distributions; it is a single-case study of one platform, so its system-side findings support analytic rather than statistical generalisation; and, because the source papers' test periods predate the present price regime, the forecasting comparison is framed in relative terms (rMAE and rankings) rather than as a comparison of absolute errors.

1.5 Report structure

Chapter 2 reviews the EPF and MLOps literatures and the evaluation methodology this work draws on. Chapter 3 specifies the three forecasting models, their features, the recalibration scheme, and the metrics and significance tests used to compare them. Chapter 4 documents the production system around those models, organised around Zarour's nine practices. Chapter 5 sets out how the system is evaluated — which tests are applied and why. Chapter 6 reports the forecasting comparison, and Chapter 7 the system evaluation. Chapter 8 answers the three research questions and draws the two threads together, and Chapter 9 concludes and identifies future work.

2 Background and Related Work

This chapter reviews the two literatures the thesis joins — electricity price forecasting and MLOps — together with the evaluation methodology it borrows from empirical software engineering. Section 2.1 covers day-ahead EPF and its statistical character; Section 2.2 the open-access benchmark that anchors the modelling thread; Section 2.3 the Transformer approach that supplies the third model; Section 2.4 the MLOps practices and maturity models that structure the system thread; and Section 2.5 the instruments used to evaluate the platform.

2.1 Day-ahead electricity price forecasting

In a day-ahead market, participants submit bids for each delivery period of the following day and a uniform clearing price is set, once per day, at the intersection of aggregated supply and demand within each bidding zone. The clearing price is set by the marginal generator, so it inherits the cost structure of the generation mix and the topology of cross-border exchange. This mechanism produces a price series with pronounced multi-scale seasonality (daily, weekly, and annual), heavy tails, and sudden spikes driven by scarcity or congestion. As wind and solar capacity has grown, two further features have become routine: sharp drops, and outright negative prices, when inflexible must-run generation coincides with high renewable infeed and low demand. Weron's review [1] surveys the statistical and computational-intelligence methods developed for this problem, and Nowotarski and Weron [2] extend the picture to probabilistic forecasting; both emphasise that EPF methodology — not merely model choice — determines whether reported gains are real.

2.2 The Lago et al. open-access benchmark

Lago et al. [3] address the reproducibility problem in EPF directly, contributing an open-access benchmark comprising five market datasets (Nord Pool, PJM, and the EPEX zones BE, FR, and DE), two reference models, and the `epftoolbox` library that standardises data splits and evaluation. The first reference model, LEAR (Lasso Estimated AutoRegressive), fits an L^1-regularised linear model over a parameter-rich feature space of price lags, exogenous forecasts, and calendar dummies; the Lasso performs feature selection and coefficient estimation jointly, which keeps the rich input space tractable. The second is a deep neural network — a multi-layer perceptron whose architecture and inputs are tuned per dataset. Despite its simplicity, LEAR matched or outperformed the deep alternative when evaluated rigorously over long test periods, which is why it serves as the modelling foundation of this thesis.

Two methodological contributions of the benchmark are reused here directly. First, models are recalibrated on a rolling window and their forecasts averaged across an ensemble of window lengths, rather than trained once. Second, the benchmark argues for the relative mean absolute error (rMAE), which normalises the model's error by that of a naive baseline so that results remain comparable across periods and markets, and for formal significance testing of accuracy differences via the Diebold–Mariano [7] and multivariate Giacomini–White [8] tests. The benchmark also adopts the inverse-hyperbolic-sine (asinh) variance-stabilising transform of Uniejewski et al. [9], which compresses spikes while remaining defined for negative prices.

2.3 Transformer approaches to EPF

Attention-based architectures have more recently been applied to EPF. Llorente and Portela [6] propose an encoder-only Transformer that takes two inputs — a sequence of the previous days' prices and the day-ahead exogenous forecasts — embeds and attends over the price sequence, and combines the result with the exogenous representation through a final multi-layer perceptron. Benchmarked on the same open-access datasets, it provides the third model family compared in this work. Notably, the authors evaluate without retraining inside the test period, applying only a renormalisation step between days; this thesis instead places the Transformer in the same rolling-recalibration pipeline as the other models, a deviation documented in Chapter 3.

2.4 MLOps: practices and maturity

MLOps applies DevOps principles — automation, version control, continuous delivery, and monitoring — to the particular demands of machine-learning systems, where data and models, not only code, evolve over time. Zarour et al. [4] consolidate the field in a systematic literature review that identifies nine recurring best practices: automating CI/CD, orchestrating workflows, ensuring reproducibility, versioning, collaboration and communication, continuously training and evaluating models, tracking and logging ML metadata, continuous monitoring, and implementing feedback loops. These nine practices provide the organising structure for the system chapter (Chapter 4).

The same review surveys five MLOps maturity models, all descended from the CMM/CMMI lineage: the Google three-level model, the Microsoft and GigaOm five-level models, the four-level model of John, Olsson and Bosch, and the five-level model of Mateo-Casalí et al. This work positions the platform against the Google model [10], whose three levels (manual process; ML-pipeline automation; CI/CD-pipeline automation) match the granularity of a single-host platform; the choice is justified in Chapter 5. The closest domain application is Subramanya et al. [5], who build an MLOps pipeline for an electricity-market forecasting task and evaluate it as a case study — the precedent this thesis follows in evaluating its own platform in its real operating context.

2.5 Evaluating machine-learning systems

Because the platform is a single artefact rather than a sample, its system-side evaluation follows the single-case study methodology of Runeson and Höst [11], which supports *analytic* generalisation — extending findings to cases sharing the platform's characteristics — rather than statistical generalisation to a population. Within that frame, several established instruments are composed. The ML Test Score of Breck et al. [12] is a 28-item rubric across four sections (data, model development, ML infrastructure, and monitoring) whose final score is the minimum of the four section sums, so the weakest area bounds the result. Reproducibility is treated in two senses: Gundersen and Kjensmo [13] characterise degrees of reproducibility in AI research, while the ACM Artifact Review and Badging policy [14] supplies the standard vocabulary — Available, Functional, Reusable, and Reproduced — for claims about a research artefact. Delivery performance is measured with the DORA metrics popularised by Forsgren et al. [15], of which deployment frequency and lead time for changes capture throughput. Finally, the decision of whether drift should trigger retraining is informed by the cost-aware analysis of Mahadevan and Mathioudakis [16], who show that triggered retraining earns its keep mainly when retraining is expensive. How these instruments are applied, and why each was chosen, is the subject of Chapter 5.

3 Methods — Forecasting Models and Evaluation

This chapter specifies the forecasting models that the platform serves and the protocol by which they are compared. Three model families are studied: the Lasso Estimated AutoRegressive ensemble (LEAR) and the deep neural network (DNN) of Lago et al. [3], and the encoder-only Transformer of Llorente and Portela [6]. Each is reimplemented faithfully and run on recent day-ahead data for two European bidding zones, DK1 (Nord Pool) and DE (EPEX). The chapter first states what ”faithful replication” means and why the comparison is framed in relative rather than absolute terms (§3.1), then describes the data and feature construction (§3.2), the three models as implemented (§3.3), the rolling recalibration and backtesting scheme (§3.4), the evaluation metrics and significance tests (§3.5), the hyperparameter-tuning protocol (§3.6), and finally the documented deviations from the source papers (§3.7).

3.1 Replication framing

The error magnitudes reported in the source papers were obtained on 2016–2018 test periods whose price level and volatility differ sharply from the 2025–2026 regime studied here; an absolute mean-error figure is therefore not comparable across the two eras. The comparison this work makes is consequently threefold. First, *methodology replication*: the same model specifications, feature sets, recalibration scheme, metrics, and statistical tests as the papers, executed on current data. Second, *relative accuracy*: the relative mean absolute error (rMAE), which Lago et al. [3] designed precisely for cross-period and cross-market comparison, together with the ordinal ranking of the models against the rankings the papers report. Third, *head-to-head significance*: paired Diebold–Mariano [7] and Giacomini–White [8] tests across the three models and a weekly-naive baseline, per market, on identical test days. Because all six experimental runs (three models $\times$ two markets) share a single test window, the paired tests are valid: every pair of models is compared on the same delivery days.

3.2 Data and features

Table 3.1: Datasets. Both markets span 2022-03-10 to 2026-05-31 (ENTSO-E Transparency Platform).

Market	Resolution	Exogenous (paper-exact)	Paper analogue
DK1	hourly (24)[a]	TotalLoad + Wind (on+offshore)	Nord Pool: load + wind [3]
DE	hourly (24)[a]	TotalLoad + Renewables (wind+solar)	EPEX-DE: load + wind&solar [3]

[a] Both markets clear in 15-min blocks (DE throughout; DK1 since 2025-10-01); prices and exogenous forecasts are downsampled to an arithmetic hourly mean (`resample_to_hourly`).

All data are obtained from the ENTSO-E Transparency Platform and stored in ClickHouse, from which the preprocessor loads a price series and a configurable set of exogenous forecast series for each market. Following Lago et al. [3], Eq. 3, the feature vector for each

delivery day combines autoregressive price lags at $d{-}1$, $d{-}2$, $d{-}3$, and $d{-}7$; exogenous forecasts at $d0$, $d{-}1$, and $d{-}7$; and seven day-of-week dummies. For a day discretised into n delivery periods this yields

$$\underbrace{4n}_{\text{price lags}} + \underbrace{2 \cdot 3n}_{\text{two exogenous} \times 3 \text{ lags}} + \underbrace{7}_{\text{weekday}} \tag{3.1}$$

features, i.e. 247 at hourly resolution ($n{=}24$) and 967 at quarter-hourly resolution ($n{=}96$). The exogenous set is restricted, per market, to exactly the two signals the papers use: for DK1 the zonal load forecast and an aggregated wind forecast (onshore plus offshore); for DE the zonal load forecast and an aggregated renewables forecast (onshore plus offshore wind plus solar). The aggregates are formed at load time by summing the raw ENTSO-E curves: the member curves are inner-joined on their timestamps before summing so that a partial sum can never occur, and all exogenous columns are then inner-joined together, so the coarsest available series anchors the common time grid.

The target undergoes the asinh variance-stabilising transform of Uniejewski et al. [9], which compresses the heavy tails and price spikes characteristic of day-ahead series while, unlike a logarithm, remaining defined for the negative prices that occur under high renewable infeed. Both markets now clear in fifteen-minute blocks — DE throughout the sample and DK1 since 2025-10-01 — but prices and exogenous forecasts are downsampled to an arithmetic hourly mean (24 periods per day) before features are constructed. This keeps the two markets comparable with each other and with the hourly reference benchmarks of Lago et al. [3] and Llorente and Portela [6]; modelling DK1 natively at 96 periods is in any case infeasible, as it has under one month of quarter-hourly history before the test window. Finally, ENTSO-E lacks DK1 wind forecasts for a small number of days inside the sample; affected days are dropped identically from every DK1 model, so the paired significance tests remain valid.

3.3 Models

Table 3.2 summarises the three model setups, which are identical across both markets except for the market-specific exogenous aggregate and the per-market DNN hyperparameters.

Table 3.2: Model setups (one row per model; identical across both markets unless noted).

Model	Specification	Calibration	Refit
LEAR	One Lasso per delivery period; λ by `LassoCV` (time-series CV)	ensemble over 56 / 84 / 1092 d	daily
DNN	MLP; architecture HPs by Optuna TPE (ex-ante validation); 4-seed ensemble at serving, single seed in backtest	730 d	daily
Transformer	Encoder stack per [6]; paper Nord Pool hyperparameters	1092 d	weekly

3.3.1 LEAR

LEAR is a direct multi-period regressor: one independent Lasso model is fitted per delivery period, so an hourly market trains 24 separate Lasso estimators, each predicting a single hour from the full feature vector of Eq. (3.1). The L1 penalty performs simultaneous shrinkage and feature selection, which is what makes the parameter-rich input space

tractable. The regularisation strength λ is selected automatically per period by `LassoCV` with a five-fold `TimeSeriesSplit` cross-validation (maximum 5000 coordinate-descent iterations), the configuration denoted `alpha="auto"`. LEAR carries no learned ensembling of its own; the ensemble averaging it benefits from is supplied by the multi-window recalibration scheme of §3.4.

3.3.2 Deep neural network

The DNN is a multi-layer perceptron with two hidden layers that maps the feature vector to all n delivery periods in a single forward pass (a multiple-input–multiple-output design). It is trained to minimise the mean absolute error with the Adam optimiser, with optional L1 weight regularisation, optional batch normalisation, and early stopping (the best weights, by validation loss, are restored from memory rather than from disk). The serving prediction is the arithmetic mean of four replicates trained from distinct random seeds, following the ensemble recommendation of Lago et al. [3], §4.4. This four-seed averaging is applied only on the production serving path; the rolling backtest that produces the Chapter 6 results trains a single network per recalibration at a fixed seed, so the reported DNN metrics carry the seed variance of one draw (revisited in §6.5). The six architecture hyperparameters — hidden-layer width, activation, dropout, learning rate, the batch-normalisation flag, and the L1 penalty — are selected per market by the tuning protocol of §3.6; Table 3.3 lists the values in force.

Table 3.3: Tuned DNN hyperparameters per market. Selected by the protocol of §3.6 on the ex-ante validation window; the architecture-only search is re-run whenever the feature set changes, so these values are refreshed by the paper-exogenous re-tune that accompanies the final benchmark.

Market	Width	Activation	Dropout	Learning rate	Batch norm	L1 penalty
DK1	191	tanh	0.203	4.8×10^{-4}	no	6.8×10^{-8}
DE	373	relu	0.204	6.9×10^{-4}	yes	7.1×10^{-7}

3.3.3 Transformer

The Transformer follows Llorente and Portela [6] and is encoder-only with two parallel input branches. The first branch takes the past-price sequence of the previous $m{=}14$ days, each day a vector of n period prices; each day is linearly embedded to a 128-dimensional vector, a learnable positional encoding is added, and the sequence is passed through a stack of six Transformer encoder layers (eight attention heads, feed-forward dimension 1024, dropout 0.2), of which the final position is taken as the price representation. The second branch linearly embeds the day-ahead exogenous forecasts. The two representations are concatenated and mapped by a single-hidden-layer perceptron (width 128) to the n output periods. The model is trained with Adam at a learning rate of 10^{-4}, batch size 32, for up to 200 epochs with early stopping (patience 25) on an internal validation split, under a step learning-rate schedule. The number of exogenous inputs is fixed at two to match the paper specification, and the remaining hyperparameters follow the paper's reported architecture and search ranges rather than a per-market optimum — the 128-dimensional embedding lies within the paper's search range but below its Nord Pool optimum of 512. Per-market Transformer tuning is left as future work (§3.7). Because the wide feature vector this model consumes is interpreted positionally rather than by column name, it is produced by a dedicated feature pipeline (`default_transformers_for_transformer`) that lays out the $m{\times}n$ price block followed by the exogenous block in a fixed order.

3.4 Recalibration and backtesting

All three models are evaluated by a single walk-forward backtesting engine that reproduces the rolling-recalibration scheme of Lago et al. [3]. For each delivery day in the test window, and for each calibration-window length in the model's ensemble, the engine slices the training data to the most recent window of days strictly preceding the target day, fits the feature and target scalers on that slice alone, trains the model, and predicts the target day; the day's forecast is the arithmetic mean of the predictions across the calibration windows. To bound compute, each window keeps a cached fitted model and is refitted only when the number of days since its last fit reaches the configured refit interval — daily for LEAR and the DNN, weekly for the Transformer, following the respective papers. LEAR uses the paper ensemble of 56-, 84-, and 1092-day windows (the 1456-day window of the original specification is dropped because it predates the available data); the DNN uses a single 730-day window; and the Transformer uses a single 1092-day window, which was verified to leave enough usable sequence rows before the test start in both markets (at least 1092 in both; 1265 measured for DE). The train/test split is date-based: training data are all rows strictly before the test start date, and no validation holdout is carved out of the backtest itself (the validation window of §3.6 is used only by the tuning command).

3.5 Evaluation protocol

The test window runs from 2025-10-24 to 2026-05-31 (220 calendar days) and is identical for all six runs. Four accuracy metrics are reported. For N aligned prediction–actual pairs $(\hat{y}_i, y_i)$,

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|, \qquad \text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}, \tag{3.2}$$

$$\text{sMAPE} = \frac{100}{N}\sum_{i=1}^{N}\frac{|y_i - \hat{y}_i|}{\frac{1}{2}(|y_i| + |\hat{y}_i|)}, \qquad \text{rMAE} = \frac{\text{MAE}(\mathbf{y}, \hat{\mathbf{y}})}{\text{MAE}(\mathbf{y}, \hat{\mathbf{y}}^{\text{naive}})}, \tag{3.3}$$

where the naive benchmark $\hat{\mathbf{y}}^{\text{naive}}$ is the weekly-naive forecast (the realised price exactly seven days earlier), as recommended by Lago et al. [3], §5.4. The rMAE is the headline metric because, being normalised by the same-market naive error, it is the figure that can be compared across periods and markets.

Statistical significance is assessed with the multivariate Giacomini–White test [8, 3]. For each ordered pair of models (A, B) the daily loss differential

$$d_t = \sum_{h=1}^{n}\left(|e^{A}_{t,h}| - |e^{B}_{t,h}|\right) \tag{3.4}$$

is formed over the n delivery periods of day t (the L^1 norm of the errors), and the test statistic $\bar{d}/\hat{\sigma}_{\bar{d}}$ is computed with a Newey–West heteroskedasticity- and autocorrelation-consistent standard error using a lag of $\lfloor T^{1/4} \rfloor$ over the T test days. The univariate Diebold–Mariano test [7] is reported as a complement. Models are compared on the test days that all of them produced, since the different feature pipelines drop gap and daylight-saving days slightly differently across model families. Finally, computation cost is reported as the training time per recalibration, read from the experiment tracker, as the analogue of Lago et al. [3], Table 4.

3.6 Hyperparameter tuning

The DNN is the only model whose hyperparameters are searched; LEAR selects λ internally by cross-validation, and the Transformer reuses the paper optimum. Tuning uses Optuna's Tree-structured Parzen Estimator over a six-dimensional architecture-only search space: hidden-layer width (16–512, log scale), activation (one of relu, tanh, elu, softplus, sigmoid), dropout (0–0.5), learning rate (10^{-5}–10^{-1}, log scale), the batch-normalisation flag, and the L1 penalty (10^{-8}–10^{-2}, log scale). The objective is the mean MAE over the validation window of 2025-01-01 to 2025-10-23 — the 42 weeks immediately preceding the test window, kept strictly ex-ante per Lago et al. [3], §4.3.2 — evaluated under the same walk-forward engine used for the backtest. Every trial is persisted to the `algo.hyperparam_trials` table so that a search resumes across sessions. The ten binary feature- and selector-toggles of Lago et al. (§4.3.3), together with their `preprocessing` and `init_method` choices, are deliberately omitted: the preprocessor and model constructors do not expose the corresponding switches, so including them in the search space would create inert dimensions that consume the trial budget without affecting the model.

3.7 Deviations from the source papers

The replication is faithful in specification but departs from the source papers in a small number of documented respects, summarised in Table 3.4. Each deviation is driven either by the length of the available data history (which begins on 2022-03-10) or by a compute budget, and none changes the model class or the feature definition.

Table 3.4: Documented deviations from [3] and [6].

Aspect	**Paper**	**This work (reason)**
Test length	$\geq$ 1–2 years	31 weeks (price history starts 2022-03; validation and calibration consume the rest)
DNN tuning budget	1500 TPE iterations	30 trials (compute; the optimum is stable early in prior studies)
DNN search space	architecture + feature selectors	architecture only (feature set fixed to the paper specification)
LEAR windows	56 / 84 / 1092 / 1456 d	56 / 84 / 1092 d (1456 predates the data)
DNN window	4 years	730 d (data and compute constraint)
λ selection	LARS+AIC, then coordinate-descent refit	`LassoCV` time-series CV (standard; close accuracy)
Market resolution	hourly (paper era)	both markets now 15-min; downsampled to an hourly mean for cross-market and cross-paper comparability
Transformer pipeline	train-once + renormalise	calibration window + weekly refit (consistency with the other models)
Transformer HPs	tuned per dataset	paper architecture / search-range defaults reused (per-market tuning is future work)

4 System Implementation

4.1 Scope and approach

4.1.1 Scope of this chapter

This chapter documents the *system* built around the forecasting models: the orchestration layer, infrastructure choices, observability stack, and CI/CD pipeline that together enable the platform to run reliably in production. It does not cover the modelling research itself — LEAR hyperparameters, DNN architecture, the asinh price transformation, or the Diebold–Mariano and Giacomini–White statistical tests (Chapter 3) — except where a modelling decision directly motivates or constrains an MLOps practice. Operational how-to material such as running backfills, adding new bidding areas, or re-deploying Prefect flows is maintained in the repository's `RUNBOOK.md` and `CONTRIBUTING.md` and is not repeated here.

4.1.2 Methodology — Zarour (2025) as structuring lens

The chapter is organised around the nine MLOps practices identified in the systematic literature review by Zarour et al. [4]: automating CI/CD, orchestrating workflows, ensuring reproducibility, versioning, collaboration and communication, continuously training and evaluating models, tracking and logging ML metadata, continuous monitoring, and implementing feedback loops. Each practice section follows a fixed shape: (i) Zarour's own framing of the practice and what it requires of a production ML system; (ii) the concrete implementation found in algo-repo; (iii) the technology chosen and the alternatives that were considered and rejected; and (iv) the design rationale given the single-host MVP scope of the platform. This structure makes it straightforward to assess which practices are fully implemented, partially implemented, or deliberately deferred, and to trace every technology choice back to an explicit justification. The same nine-practice vocabulary structures the maturity assessment in Chapter 7.

4.2 System at a Glance

Before examining each Zarour practice in detail, this section presents the platform as a whole: its host-level topology, the end-to-end data plane, the complete inventory of scheduled deployments, and the technology choices that underpin every layer.

4.2.1 Host-level architecture

The platform runs entirely within a single Docker Compose stack on one dedicated host, keeping operational complexity proportionate to the MVP scope. Five long-lived services form the persistent backbone of that stack: ClickHouse, which stores all time-series data and ML artifacts; the Prefect server, which exposes the orchestration API and web UI; the Prefect worker, which polls for scheduled runs and manages a Docker work pool; Loki, which receives structured log streams from Python via `python-logging-loki`; and Grafana, which visualises both ClickHouse metrics and the Loki log stream through file-provisioned dashboards. When the Prefect worker dispatches a scheduled flow run it spawns an ephemeral container from the `algo-repo:latest` image; that container reads from and writes to ClickHouse, calls the ENTSO-E Transparency Platform API, and emits structured logs back to Loki before being automatically removed on completion.

4.2.2 Data plane

Data enters the platform through the ENTSO-E Transparency Platform API, where four fetch functions retrieve energy prices, total-load forecasts, generation forecasts, and wind

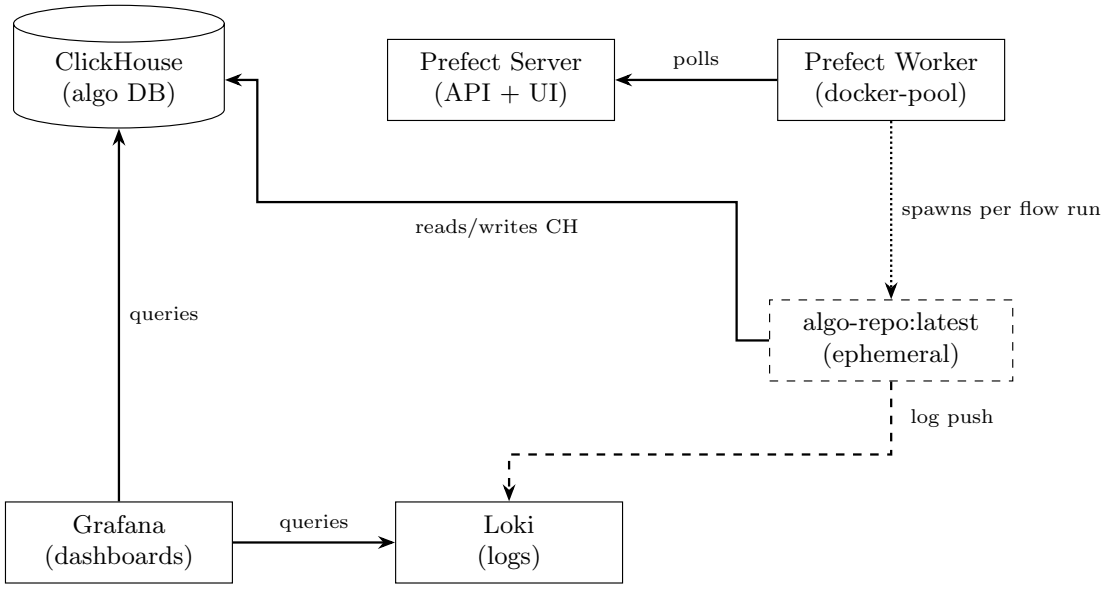


Figure 4.1: Host-level architecture. A single Docker Compose host runs ClickHouse, the Prefect server and worker, Loki, and Grafana. Flow runs spawn ephemeral `algo-repo:latest` containers that read ClickHouse, call ENTSO-E, and emit logs back to Loki.

and solar forecasts for each configured bidding area. The raw XML responses are parsed by typed mapper functions that produce `Observations` and `Forecast` domain objects, which are then ingested into ClickHouse tables `algo.timeseries` and `algo.forecasts` via Arrow-batch inserts. Both tables use `ReplacingMergeTree` engines, which deduplicate lazily on merge; read paths therefore either append the `FINAL` keyword to their queries or call `drop_duplicates` on the loaded DataFrame to ensure correctness. A `DataPreprocessor` loads this data, pivots it to a daily-wide feature matrix through a configurable sequence of `FeatureTransformer` objects, and feeds it to either `BacktestEngine` for offline evaluation or `LiveRunner` for day-ahead production forecasting. Predictions and trained model artifacts are persisted to `algo.experiment_runs`, `algo.experiment_predictions`, and `algo.model_artifacts`, from where the reconciliation flow back-fills actuals the following day and the drift-detection flow compares the recent price distribution against a ninety-day reference window, triggering a Slack alert and an optional automatic retraining run when significant drift is detected.

4.2.3 Operational inventory

Table 4.1 lists every Prefect deployment currently registered in the system, together with its schedule, the flow it executes, and the Zarour practice it primarily realises.

4.2.4 Technology stack

The table below records each tool's role, whether it was already present at the start of the MLOps build-out or was added during it, and the one-line rationale for the choice.

4.3 Zarour's Nine MLOps Practices — Implementation

The nine sections that follow map every MLOps practice identified by Zarour et al. (introduced in Chapter 2) to a concrete subsystem in algo-repo. Each follows the same shape: the practice's framing, the implementation found in this repository, the technology chosen and its rejected alternatives, and the design rationale scoped to a single-host MVP. Where

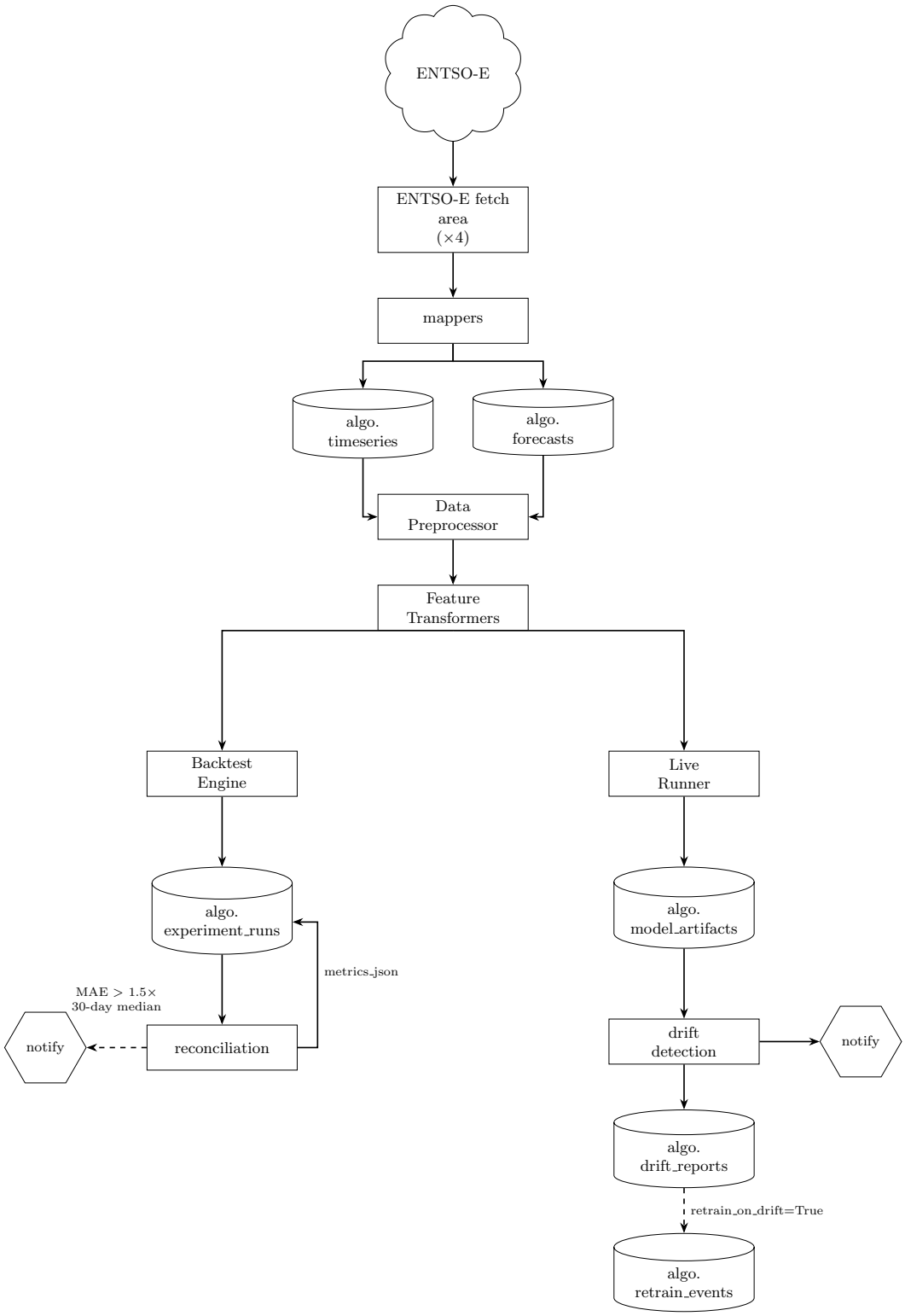


Figure 4.2: Data plane. ENTSO-E data flows through typed mappers into ClickHouse, where the forecasting pipeline reads it, trains and evaluates models, and writes predictions and artifacts back. Reconciliation and drift detection close the loop on a daily cadence.

a practice is only partially realised, the limitation is stated explicitly and cross-referenced to the maturity assessment in Chapter 7.

4.3.1 P1 — Automating CI/CD

Automating continuous integration and continuous delivery, as characterised by Zarour et al., requires that every proposed change be automatically validated before it reaches the main branch, and that an accepted change then propagate to the running system without manual intervention [4]. The practice prevents the accumulation of silent regressions and keeps the repository and the deployed host continuously convergent.

CI/CD in algo-repo is a layered system of gates: three GitHub Actions workflows protect every pull request, and one deploy workflow propagates every merge to `main` to the production host. On the integration side, `pr_test.yml` runs the full pytest suite, which mirrors the `src/` layout and covers domain models, XML mappers, ClickHouse ingest, and the forecasting pipeline including the LEAR smoke test in `tests/price_forecasting/test_smoke.py`; `pr_ruff-lint.yaml` runs `ruff check` for linting and import hygiene; and `pr_smoke.yml` builds the `algo-repo:latest` image, starts a Compose stack with ClickHouse and a Prefect server and worker, and executes an end-to-end daily-forecast and reconciliation run against synthetic data. A path-filter step skips the expensive smoke job when a push touches only files outside the gated paths, so documentation-only commits pass the required check without waiting for a multi-minute Docker build.

On the delivery side, `cd_deploy.yml` fires on every merge to `main`: it joins the production host over the Tailscale network, pulls the merged code, runs ClickHouse migrations, rebuilds `algo-repo:latest` with the commit SHA baked in via `--build-arg GIT_SHA=$(git rev-parse HEAD)`, reruns both `deploy_ingestion.py` and `deploy_forecast.py` to register updated Prefect deployments, and finally triggers `daily-forecast-dk1_lear` and polls until it completes, confirming that the freshly deployed image produces a valid forecast on the live host. A parallel `reload_grafana` job restarts Grafana so that file-provisioned dashboard and alert-rule changes take effect, and a separate `main_cache_warm.yml` workflow pre-populates the Actions cache used by the next PR's Docker build.

The deploy host accepts inbound connections only over the Tailscale mesh, so the public internet sees no open SSH port. A publicly reachable SSH endpoint, a bastion host, or a VPN appliance would each either expose a port to address enumeration or require a separately maintained jump host; Tailscale replaces both with a WireGuard mesh in which every device authenticates with the coordination server before a packet is accepted. The GitHub Actions runner joins the tailnet for the duration of the workflow via `tailscale/github-action`, using an ephemeral `TS_AUTHKEY` scoped to the CI device tag, and the connection is torn down when the runner exits, so no permanent network path to the host is ever open.

GitHub Actions was chosen over Jenkins or CircleCI because it is co-located with the repository and requires no separately managed runner host. Keeping the three PR workflows as separate files lets the slow smoke test be skipped or re-run in isolation without re-triggering linting; separating the PR gate from the deploy gate keeps the former enforcing correctness and the latter enforcing delivery.

4.3.2 P2 — Orchestrating workflows

Orchestrating workflows, in Zarour et al.'s framing, is the practice of expressing the recurring data and model tasks as scheduled, observable, retryable pipelines rather than as cron-driven scripts, so that failures are isolated, retried, and surfaced rather than silently lost [4].

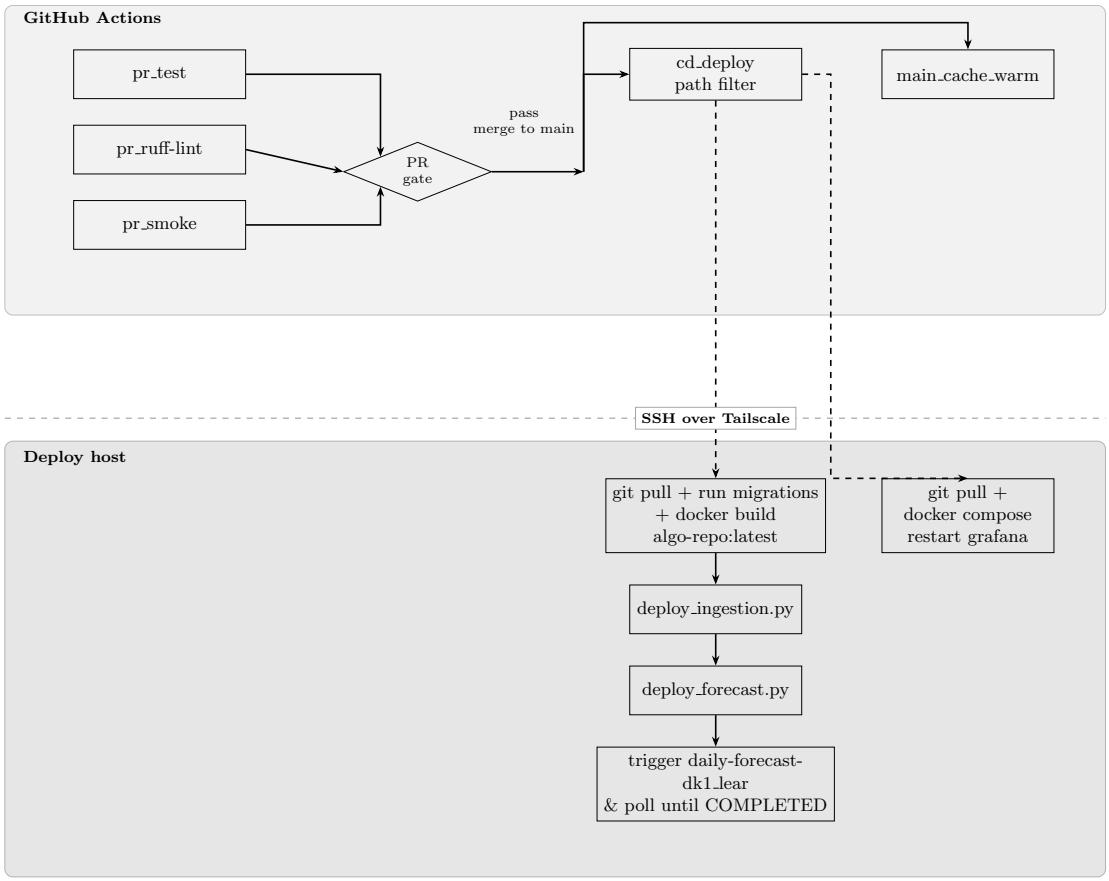


Figure 4.3: CI/CD pipeline. GitHub Actions runs the PR gate and the deploy gate; the deploy gate reaches the production host across the Tailscale link, rebuilds the image, redeploys Prefect, and triggers a smoke forecast.

All recurring work on the platform is expressed as Prefect flows executed by a Docker work pool: `entsoe_backfill_flow` (ingestion), `daily_forecast_flow` (refit and forecast), `reconcile_predictions` (day-after actuals), and `drift_detection_flow`. Scheduling is declarative: each deployment in Table 4.1 attaches a cron schedule to a flow, and the worker spawns an ephemeral `algo-repo:latest` container per run. The ingestion flow is parameterised by area — one flow definition, `entsoe_backfill_flow(area)`, with one deployment per bidding zone — so adding a market is a one-line deployment change with no flow-code edit. Within a flow, granularity is one task per data type (prices, load, generation, wind and solar); tasks run sequentially to respect the ENTSO-E rate limit and each carries `retries=2` with a 60-second delay, so one data type failing does not abort the others. The ingestion flow re-ingests a rolling 48-hour lookback window on every run; because both ClickHouse tables are `ReplacingMergeTree`, re-ingesting overlapping data is always safe.

A deliberate design choice keeps the fetch-and-insert logic in a shared service layer, `data_cli.entsoe.services`, with no Prefect dependency, so that both the Typer CLI (`backfill.py`) and the Prefect flow (`ingestion.py`) import the same code path rather than duplicating it. Prefect was chosen over Airflow and Dagster: at single-host scale its Python-native flow definitions and Docker work pool are sufficient, and it avoids the operational weight of an Airflow scheduler or a Dagster daemon and database.

4.3.3 P3 — Ensuring reproducibility

Reproducibility in the context of MLOps requires that a past model result can be reconstructed from the same code version, data window, hyperparameters, and *runtime environment*, without relying on implicit host state [4]. Zarour et al. identify it as one of the most frequently cited challenges in practitioner surveys, and treat containerisation of the runtime as one of its principal enablers.

The platform addresses reproducibility along three axes. First, the runtime is fully containerised and the topology is infrastructure-as-code: the entire operational stack is declared in `docker-compose.yml` (the five long-lived services, their volume mounts, environment sources, and network), flow-run containers are built from `Dockerfile` and the worker from `Dockerfile.worker`, and Grafana's provisioning, the alert rules, and the Prometheus scrape config are all file-mounted from the repository, so the infrastructure can be rebuilt from the repository alone with no hand-applied host commands. Second, every row in `algo.experiment_runs` carries `git_sha`, `data_start_utc`, and `data_end_utc`, a minimal provenance triple that fixes both the code version and the data window of any run; the `GIT_SHA` is baked into the image at build time via `--build-arg GIT_SHA=$(git rev-parse HEAD)` rather than resolved at runtime, because the deploy host holds only the built image and not the source tree, and local runs emit an empty string so a dirty tree never stamps a misleading SHA. Third, the modelling pipeline is itself deterministic given the config: calibration windows and the `AsinhScaler` target transform are fixed in the committed configs, and the DNN ensemble averages `n_dnn_ensemble` replicates with distinct, enumerated random seeds, so the trained ensemble is fully described by the config and a deterministic seed sequence.

Docker Compose was chosen over Kubernetes for the container runtime: at single-host scope, a control plane, etcd, and Helm charts are unjustified overhead (Kubernetes is a deferred tool, §4.5). Baking the SHA into the image rather than resolving it at container start makes every `algo.experiment_runs` row self-describing under a deployment model that has no source checkout on the host.

4.3.4 P4 — Versioning

Versioning is the foundational practice under which all code, data schemas, and configuration that affect model behaviour are tracked in source control, enabling reproducible reconstruction of any past system state [4]. Zarour et al. treat it as a prerequisite for every other practice, because without a shared history of code and configuration changes, claims about reproducibility or rollback cannot be substantiated.

The platform uses Git with GitHub as the remote, and versions every artefact that can affect a model run. Python experiment configs in `configs/` are first-class Git files whose history records every calibration-window or scaler change; ClickHouse schema changes live in numbered SQL migration files under `src/infrastructure_clickhouse/migrations/` that are auto-discovered in sorted order by `run_migrations.py` and never modified retroactively; and Grafana dashboard definitions (JSON under `grafana/dashboards/`) and alert rules (`grafana/provisioning/alerting/rules.yaml`) are file-provisioned, never edited through the UI, and subject to code review before taking effect.

DVC was evaluated for data versioning and rejected: ENTSO-E data is replayable in full from the public API, and combining ClickHouse's `ReplacingMergeTree` engine with the `git_sha` column on `algo.experiment_runs` provides sufficient provenance without maintaining a separate binary store. Versioning every artefact that affects model output means a single `git log` spans the experiment, the infrastructure, and the alerting policy together, closing the gap between operational changes and the audit trail expected of a

production ML system.

4.3.5 P5 — Collaboration and communication

Collaboration and communication, as Zarour et al. frame it, encompasses the practices that let a team work on a shared ML system without relying on tribal knowledge: conventions files, onboarding guides, runbooks, and tooling that surfaces system events proactively rather than requiring dashboards to be polled [4]. The practice matters more in ML systems than in conventional software because the surface of implicit knowledge — calibration-window choices, scaler semantics, `ReplacingMergeTree` deduplication behaviour — is larger.

The repository carries four documentation artefacts. `CLAUDE.md` records project conventions — naming rules, CLI flag order, circular-import constraints, design rationale — written to be machine-readable by LLM coding agents as well as by human contributors. `docs/REPO_OVERVIEW.md` provides a deep reference for any reader coming in cold. `docs/RUNBOOK.md` covers manual interventions and incident triage. `docs/CONTRIBUTING.md` documents the branching policy, PR conventions, and the procedure for onboarding a new bidding area. On the communication side, drift events and accuracy-degradation alerts post to a Slack webhook via `schedules.alerting.notify`, so the team is notified of production anomalies even when nobody is watching Grafana.

Slack webhooks were chosen over PagerDuty or Opsgenie because the team is small enough that a single notification channel suffices — there is no on-call rotation that would benefit from escalation policies. The `notify` helper is a safe no-op when `SLACK_WEBHOOK_URL` is unset, so local development and CI never fail on a missing webhook. Splitting documentation between a terse, rule-dense `CLAUDE.md` for agents and a narrative `REPO_OVERVIEW.md` for humans serves two readers while keeping both files load-bearing and less prone to drifting out of date.

4.3.6 P6 — Continuously train and evaluate

Continuous training and evaluation requires that the model be retrained automatically on a regular cadence so that it reflects the most recent data distribution, and that its live predictions be continuously scored against realised outcomes [4]. Zarour et al. distinguish the scheduled, unconditional refit from the reactive drift-triggered path of P9.

The six `daily-forecast-*` deployments (LEAR, DNN, and Transformer for both DK1 and DE) execute `daily_forecast_flow` at 06:00 UTC. Each run calls `LiveRunner.run_live_with_metadata()`, which loads the latest ClickHouse data window, refits the ensemble of calibration-window models from scratch, and immediately produces a day-ahead forecast; for DNN configurations it fits `n_dnn_ensemble` seed replicates and averages them, following Lago et al. [3]. The trained models are serialised to `algo.model_artifacts`. The *evaluate* half of the practice is the reconciliation flow (§P7–P8): the day after each forecast, realised prices are joined to the stored predictions and the same MAE, RMSE, and sMAPE used in backtesting are computed on the live forecast, so offline and online evaluation use one metric implementation.

A full refit from scratch on each run was chosen over warm-start learning: the daily data increment is small relative to the calibration windows, so a clean refit is cheap and avoids accumulated approximation error. Keeping training and prediction in one flow means a single task-level retry covers both phases and the served model is always consistent with the forecast it produced.

4.3.7 P7 — Track and log ML metadata

Tracking and logging ML metadata is the practice of recording the full provenance of every model run — hyperparameters, data windows, performance metrics, and the model artefact itself — in a queryable store, so that past results can be reproduced and compared without ad-hoc file management [4].

This is handled in ClickHouse. `algo.experiment_runs` stores one row per run with `training_duration_s`, `data_start_utc`, `data_end_utc`, `model_artifact_id`, and `git_sha`, making every run self-describing; `algo.experiment_predictions` stores one row per period per day for reconciliation; `algo.model_artifacts` stores base64-encoded joblib blobs, keeping each artefact attached to its run via `model_artifact_id`; and `algo.hyperparam_trials` persists every Optuna trial, enabling resumable search across sessions. On the research surface, W&B serves two projects: `algo-research` for backtests, ablation sweeps, and Optuna sweeps, and `algo-prod` for production reconciliation — one continuous run per `(area, config_name)` pair, one step per reconciled day. `WANDB_MODE=disabled` no-ops the tracker in CI and local development.

A filesystem model store was rejected: storing artefacts as blobs in `algo.model_artifacts` keeps them attached to the experiment row rather than in a separately managed path. MLflow was evaluated and deferred until the experiment-config count exceeds ten; at the current scale ClickHouse and W&B together cover the audit-log and research-surface roles without adding a service to the Compose stack. Using one OLAP store for predictions, metrics, and artefacts makes day-after reconciliation a single query, and the `ReplacingMergeTree(created_utc)` engine makes the reconciliation update a safe re-insert rather than a mutable `UPDATE`.

4.3.8 P8 — Continuous monitoring

Continuous monitoring requires instrumenting the production system across at least three axes — data quality, model performance, and system health — so that degradation is detected automatically and surfaced without manual inspection [4].

Structured log events from every Prefect flow and infrastructure module are pushed directly to Loki via the `python-logging-loki` handler in `infrastructure_clickhouse.logging_setup`, with no Promtail sidecar. Three Grafana dashboards auto-provision from `grafana/dashboards/`: the Data Quality dashboard covers coverage and freshness of `algo.timeseries` and `algo.forecasts`; the Operational dashboard presents the Loki log stream filtered by `job=algo-repo`; and the System Health dashboard is backed by Prometheus, which scrapes ClickHouse's native `/metrics` endpoint, Grafana's `/metrics` endpoint, and itself on a 15-second cadence and retains 120 days of series. Model-quality metrics — MAE, RMSE, and sMAPE per reconciled day, per area, per config — do not live in Grafana; they are owned by the W&B `algo-prod` project, which the dual-write tracker in `observability_wandb` updates from `schedules.reconciliation`. The split is deliberate: Grafana hosts platform-and-data-health concerns, W&B hosts model-quality concerns, and both read from the same ClickHouse tables.

File-based alert rules in `grafana/provisioning/alerting/rules.yaml` define four conditions: `data-staleness` fires when `algo.timeseries` has received no new row in twelve hours; `drift-detected-today` fires when any row in `algo.drift_reports` for the current day carries `drift_detected = 1`; `reconciliation-missed` fires when no `reconcile-predictions` run has completed in the last 36 hours; and `clickhouse-down` fires when the Prometheus `up{job="clickhouse"}` liveness metric is zero. In addition, `schedules.reconciliation._maybe_alert_accuracy_degradation` posts a Slack notification whenever today's MAE exceeds 1.5 $\times$ the 30-day median, computed from `algo.experiment_runs` via `queries.get_recent_acc`

Loki was chosen over the ELK stack because it is a single binary with substantially lower memory and operational overhead, proportionate to a single-host deployment, and Promtail was rejected in favour of direct Python push, which removes a moving part from the logging pipeline. Prometheus sits alongside Loki as the metrics tier to Loki's logs tier, with Grafana the single dashboard and alerting surface above both. The three monitoring axes — data, model, and system — match Zarour's P8 decomposition and ensure an alert is raised at whichever layer first shows a problem.

4.3.9 P9 — Implementing feedback loops

The feedback-loop practice requires that the production system detect when its inputs or outputs have shifted sufficiently to degrade prediction quality and respond automatically, rather than waiting for a human to notice on a dashboard [4]. Zarour et al. identify the absence of an automated feedback loop as a common maturity gap.

Drift detection runs in-process inside the `drift-detection-dk1` and `drift-detection-de` deployments via Evidently's `DataDriftPreset`, which compares a 90-day reference window against a 7-day current window on each area's price series and auto-selects the statistical test by sample size (a Kolmogorov–Smirnov test for small samples, the Wasserstein distance for larger ones). Drift results — score, threshold, the name of the test applied, and the boolean `drift_detected` flag — are written to `algo.drift_reports`, and a detected event posts a warning to Loki and a Slack notification via `schedules.alerting.notify`. When a config carries `retrain_on_drift=True`, the flow additionally enqueues a `triggered_retrain_flow`, which writes a `running` audit row to `algo.retrain_events`, calls `daily_forecast_flow` synchronously, and closes the row with a `completed` or `failed` status and a strictly later `updated_utc` so the `ReplacingMergeTree(updated_utc)` engine retains the terminal row.

Evidently was deployed in-process rather than as a separate service so the check inherits the flow's retry policy and is re-run via the Prefect UI without managing another endpoint; Arize and WhyLabs were rejected as cloud-hosted dependencies unjustified at single-host scope. Making the retrain trigger conditional on `retrain_on_drift` keeps every retraining event auditable: each triggered run links to the specific `drift_report_id` that caused it, so the full chain from detected drift to updated model is recoverable from ClickHouse alone. Whether drift *should* trigger retraining at all, given the cheap daily refit, is examined in Chapter 7.

4.4 Cross-cutting: the reproducibility–monitoring link and MVP scope

4.4.1 GIT_SHA propagation as the reproducibility–monitoring link

Every row that `cd_deploy.yml` produces in `algo.experiment_runs` carries the same `git_sha` baked into the image during the deploy-gate build step: the SHA of the exact commit merged to `main`. The Grafana Operational dashboard queries the Loki log stream filtered by `job=algo-repo` and presents structured events that include the same SHA, because every flow container is built from that image. So if the W&B `algo-prod` run for an `(area, config_name)` shows an unexpected MAE spike beginning on a particular day, the Operational dashboard shows the Loki events from the same period alongside the `git_sha` of the image that produced them, making it possible to `git log` back to the exact commit. This closes the loop between P3 (reproducibility) and P8 (monitoring): the audit trail for a production anomaly runs from the Grafana panel, through the `algo.experiment_runs` row, to a checkable commit, with no gap.

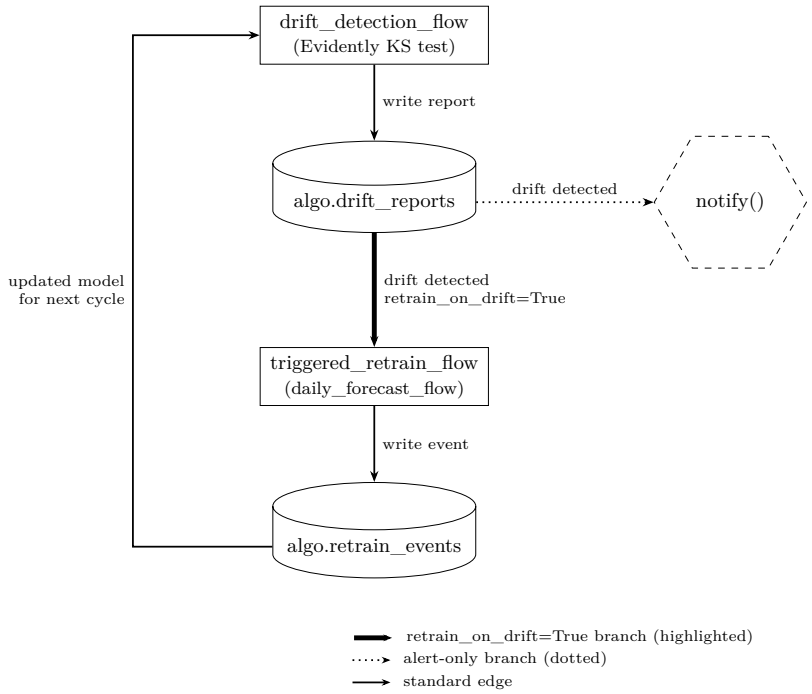


Figure 4.4: Drift-triggered retraining loop. Drift events land in `algo.drift_reports` and post to Slack; configs with `retrain_on_drift=True` additionally trigger a retraining flow that records each event in `algo.retrain_events`.

4.4.2 Why a single-host deploy is the right MVP scope

Running the entire stack on one Docker Compose host means there is exactly one failure domain: if the host goes down, all services go down together and are restored together by restarting the stack, which is the correct blast radius for a single-operator MVP. Kubernetes becomes appropriate when workloads must be distributed across hosts and the overhead of a control plane and an etcd cluster is justified by availability requirements [4]; neither holds here. Multi-host deployment would introduce a service mesh or shared storage for ClickHouse, a distributed work pool, and cross-host log aggregation — complexity that would slow iteration without improving the platform's ability to deliver daily forecasts. The single-host scope also simplifies rollback: reverting to a previous image tag is a `docker build` followed by a `docker compose restart`, with no pod drain or rolling-deployment coordination. The right adoption trigger for Kubernetes is the appearance of a second deploy host, not a speculatively chosen target architecture.

4.5 Deferred tooling and adoption triggers

Each tool evaluated and not adopted during the MLOps build-out was deferred to a specific, observable trigger rather than to an indefinite future review.

- **MLflow** — deferred until the production experiment-config count exceeds ten. At the current scale, ClickHouse and W&B together cover the audit-log and research-surface roles that MLflow would fill without adding a service to the Compose stack.
- **Kubernetes** — deferred until a second deploy host is justified by availability requirements. The single-host scope means there is exactly one failure domain; the control-plane overhead of Kubernetes is not warranted until workloads must be distributed across multiple hosts.

- **Cloud-managed model store** — deferred until DNN model weights exceed the practical limit for ClickHouse blob storage, taken as approximately ten megabytes per artifact. Below that threshold the `algo.model_artifacts` table keeps artifacts co-located with their experiment rows without requiring a separate storage service.
- **DVC** — deferred for as long as ENTSO-E data is fully replayable from the Transparency Platform API at the required resolution. The combination of `ReplacingMergeTree` deduplication and the `git_sha` column on `algo.experiment_runs` provides sufficient provenance without maintaining a separate binary data store.

Table 4.1: Operational inventory of Prefect deployments. Cron strings are UTC.

Deployment	Area / model	Cron (UTC)	Flow module	Purpose	Zarour
entsoe-backfill-dk1	DK1	0 * * * *	schedules.ingestion.entsoe_backfill_flow	Hourly ENTSO-E in-gest	P2
entsoe-backfill-dk2	DK2	0 * * * *	*(as above)*	Hourly ENTSO-E in-gest	P2
entsoe-backfill-nl	NL	0 * * * *	*(as above)*	Hourly ENTSO-E in-gest	P2
entsoe-backfill-be	BE	0 * * * *	*(as above)*	Hourly ENTSO-E in-gest	P2
entsoe-backfill-de	DE	0 * * * *	*(as above)*	Hourly ENTSO-E in-gest	P2
entsoe-backfill-fr	FR	0 * * * *	*(as above)*	Hourly ENTSO-E in-gest	P2
entsoe-backfill-pl	PL	0 * * * *	*(as above)*	Hourly ENTSO-E in-gest	P2
daily-forecast-dk1_lear	DK1 / LEAR	0 6 * * *	schedules.forecast.daily_forecast_flow	Daily refit and day-ahead fore-cast	P6
daily-forecast-de_lear	DE / LEAR	0 6 * * *	*(as above)*	Daily refit and day-ahead fore-cast	P6
daily-forecast-dk1_dnn	DK1 / DNN	0 6 * * *	*(as above)*	Daily refit and day-ahead fore-cast	P6
daily-forecast-de_dnn	DE / DNN	0 6 * * *	*(as above)*	Daily refit and day-ahead fore-cast	P6
daily-forecast-dk1_transformer	DK1 / Transformer	0 6 * * *	*(as above)*	Daily refit	P6

Table 4.2: Technology stack and rationale.

Tool	Role	Status	Why this choice
Prefect	Orchestration	Kept	Sufficient for single-host flows; lighter than Airflow or Dagster
ClickHouse	OLAP store + model artifact blob	Kept	Columnar performance, ReplacingMergeTree dedup, native Polars / Arrow path
Docker Compose	Container runtime + infra-as-code	Kept	Single-host scope; YAML matches the deploy mental model
Loki	Log aggregation	Added	Direct `python-logging-loki` push, no Promtail sidecar
Prometheus	Metrics time-series store	Added	System metrics, uptime tracking; native ClickHouse and Grafana scrape endpoints, no exporter sidecars
Grafana	Dashboards and file-based alerts	Added	File-provisioned configs survive container restarts
Evidently AI	In-process drift detection	Added	Mode 1 reports; no separate Evidently service
W&B	Research + production tracking	Added	Two projects (`algo-research`, `algo-prod`); `WANDB_MODE=disabled` in CI
GitHub Actions	CI/CD	Kept	Native to the GitHub-hosted repo, no self-hosted runner yet
Tailscale	Deploy host network access	Added	SSH to deploy host without exposing it to the public internet
Optuna	Hyperparameter search (TPE)	Added	Persisted to ClickHouse; revisited per-config rather than per-run
MLflow	Experiment registry	Deferred	Until experiment-config count exceeds ten
Kubernetes	Container orchestration	Deferred	Until multi-host deployment is justified
DVC	Data versioning	Deferred	ENTSO-E data is replayable from source; ClickHouse + `GIT_SHA` suffice

5 Evaluation Framework

5.1 Approach

This chapter sets out how the *system* is evaluated — which tests are applied and why — while the numbers themselves are reported in Chapter 7. It is concerned with operational maturity, not forecast accuracy: the modelling-side evaluation (MAE, RMSE, sMAPE, rMAE, and the Diebold–Mariano and Giacomini–White significance tests) is defined in Chapter 3 and reported in Chapter 6, so the two threads of the thesis each carry their own evaluation. Following the single-case study methodology of Runeson and Höst [11], the platform is evaluated as one system in its real operating context; there is no controlled comparison against a second platform. The honesty boundary is stated up front: the figures reported are indicative of the system over the study window, not claims about long-run production operation. A distinction that runs through the chapter, and that matters for the validity discussion, is between *objective measurements* — numbers produced by reproducible scripts — and a single *structured self-assessment*, the ML Test Score (Table 5.1).

Table 5.1: Two kinds of test used in this work.

Type	**What it produces**	**Auditable how?**
Objective measurement	A number computed by a script over code, git history, or the database	Re-run the script — same inputs, same number
Structured self-assessment	A rubric score with one-line evidence per item	Read the evidence column; agree or contest each item

5.2 Test inventory

Five tests are applied to the system: four objective measurements and one structured self-assessment, listed in Table 5.2. The remainder of the chapter takes each in turn, stating its definition, the instrument that produces it, and the reason it is included.

Table 5.2: Tests applied to the system in this work.

Test	**Type**	**Source / instrument**	**Cite**
ML Test Score	Self-assessment	28-item rubric, scored 0 / 0.5 / 1, min-of-sections	[12]
Provenance coverage	Objective	SQL aggregate over `algo.experiment_runs`	[13]
Re-run demonstration	Objective	`scripts/report/reproduce_run.py` — re-execute logged run, compare to stored prediction	[13]
DORA delivery throughput	Objective	`gh`/git mining of `cd_deploy.yml` history	[15]
Drift monitoring counts	Objective	SQL aggregate over `algo.drift_reports`	—

5.3 Why these five tests

Zarour et al.'s systematic review [4] catalogues nine MLOps practices but produces no single quantitative aggregate against which a platform can be scored. Rather than invent one, this work composes a small set of tests that each address a measurable face of one or more practices. Between them, the five tests evidence eight of Zarour's nine practices, as mapped in Table 5.3; the ninth, collaboration and communication, is qualitative by nature and is documented in the System Implementation chapter (Chapter 4) rather than measured here.

Table 5.3: Mapping of tests to the nine MLOps practices in Zarour et al. [4].

Zarour practice	**Tested by**
P1 Automating CI/CD	ML Test Score (Infra), DORA throughput
P2 Orchestrating workflows	ML Test Score (Infra)
P3 Ensuring reproducibility	ML Test Score (Infra), Provenance coverage, Re-run demonstration
P4 Versioning	ML Test Score (Data, Infra), Provenance coverage
P5 Collaboration and communication	*not measured here — qualitative, see Ch. 4*
P6 Continuously train and evaluate	ML Test Score (Model)
P7 Track and log ML metadata	ML Test Score (Model), Provenance coverage
P8 Continuous monitoring	ML Test Score (Monitoring), Drift counts
P9 Implementing feedback loops	Drift counts (re-framed as monitoring; see §7.4)

5.4 ML Test Score

The ML Test Score of Breck et al. [12] is a rubric of 28 tests across four sections — data, model development, ML infrastructure, and monitoring — each scored on the scale $\{0, 0.5, 1\}$ defined in Table 5.4. The final score is the *minimum* of the four section sums, not their total: because the four sections guard against different and non-substitutable failure modes (Table 5.5), the weakest section bounds the platform's production-readiness, and the rule rewards balance rather than breadth. The assessment is conducted as a structured self-assessment: every item is walked against the codebase, one line of evidence is recorded for it, and a score is assigned by the explicit rule. The full per-item evidence is committed to `Report/metrics/ml_test_score.md` so that each score can be checked or contested independently.

Table 5.4: Scoring rule applied to each of the 28 items.

Score	**Meaning**
1.0	Capability exists and is automated / tested / repeatable (runs in CI, runs on schedule, has a test)
0.5	Capability exists but is manual, partial, or ad-hoc
0.0	Not done, or genuinely not applicable to the domain

One honesty rule is applied throughout: no point is awarded without written evidence, and items that are genuinely not applicable to wholesale-price forecasting (for example, demographic-fairness tests) are scored zero rather than excused. That the rubric is self-administered is itself a threat to construct validity; it is named as such in §5.9 and mitigated by the recorded per-item evidence.

Table 5.5: The four sections of the rubric and what each is designed to detect.

Section (7 items each)	Failure mode it guards against
Data tests	Bad inputs reach training / serving (schema drift, missing features, costly features)
Model development tests	Models ship that are worse than a baseline, untuned, or untested on slices
ML infrastructure tests	Pipeline can't be reproduced, debugged, rolled back, or safely canary-deployed
Monitoring tests	Production failure goes undetected (silent drift, stale model, regressed accuracy)

5.5 Reproducibility tests

Reproducibility is measured in two complementary senses, following the reproducibility-degrees framing of Gundersen and Kjensmo [13] and defined in Table 5.6: *coverage*, a system-wide property asking whether every run carries enough metadata to be reproduced, and *demonstration*, a worked existence proof in which one logged run is regenerated from scratch.

Table 5.6: Two reproducibility tests.

Test	Definition
Provenance coverage	% of live runs in `algo.experiment_runs` that carry the complete triple: `git_sha`, (`data_start_utc`, `data_end_utc`), `model_artifact_id`
Re-run demonstration	Take one logged run; using only its committed config + target date, re-execute `LiveRunner`; report the max absolute difference vs the stored prediction

Both are reported because they answer different questions: coverage asks whether the system *could* reproduce any run, while the demonstration shows that it *does* for a concrete one. The deterministic LEAR configuration is used for the demonstration because, given a fixed data window, it reproduces exactly; the DNN and Transformer carry seed and ensemble stochasticity whose controlled re-run is left to future work. Against the ACM Artifact Review and Badging vocabulary [14], the artefact can be claimed Available, Functional, and Reusable by self-review; the independent "Reproduced" badge, which requires an external reviewer, is not claimed.

5.6 Delivery throughput (DORA)

Delivery performance is reported using the throughput half of the DORA metrics [15]. One deployment is defined as one successful run of `cd_deploy.yml` on `main`, and lead time is the wall-clock interval from the merge commit to the start of that run — the interval under the platform's automated control, rather than the time a feature spends in development. Table 5.7 records which DORA metrics are reported and which are deliberately withheld, with the reason in each case.

5.7 Drift monitoring counts

The drift test reports two counts over the study window, taken from `algo.drift_reports`: the number of drift reports written, and the number flagged `drift_detected = 1`. Deliber-

Table 5.7: DORA: what is measured, what is excluded, and why.

Metric	Reported?	Why / why not
Deployment frequency	Yes	Mineable from git/CI history alone [15]
Lead time for changes	Yes	Same; throughput metric
Change failure rate	*No*	Requires an incident corpus the study window does not provide
Mean time to restore	*No*	Same: needs labelled production incidents
DORA tier placement	*No*	Tier thresholds are re-clustered yearly; unsafe to assign on a short window

ately, it does *not* report a drift-to-retrain latency. Under the reframing argued in Chapter 7 (§7.4), drift detection on this platform serves monitoring and alerting rather than triggering, so a latency to retrain is not a meaningful metric. That reframing is conditional: it holds while retraining stays cheap, and would flip for expensive multi-year retraining [16] — a boundary condition surfaced again where the counts are reported.

5.8 Maturity-model positioning

The platform's per-practice maturity levels are reported in the qualitative practice-by-practice table in the Results chapter (Chapter 7, Table 7.4); this section justifies the prior question of *which* maturity model those levels are assigned against. Zarour et al. [4] review five MLOps maturity models (Table 5.8), all descended from the CMM/CMMI lineage. The platform is positioned against the Google three-level model.

Table 5.8: The five MLOps maturity models reviewed by Zarour et al. [4], Table 7. All derive from CMM/CMMI.

Model	Levels	Top level
Google	3 (0–2)	CI/CD pipeline automation
Microsoft	5 (0–4)	Full MLOps automated operations
GigaOm	5 (0–4)	Comprehensive MLOps pipeline for frequent model updates
John, Olsson & Bosch (2021)	4 (0–3)	Fully-automated model monitoring
Mateo-Casalí et al. (2022)	5 (0–4)	Optimization (pure CMMI mapping)

Google is chosen for three reasons. Its granularity matches the single-host MVP scope: three levels avoid over-claiming distinctions — such as those between Microsoft's Levels 2–4 — that a single deployment target cannot meaningfully differentiate. Its top level, CI/CD-pipeline automation, is plausibly attainable for this platform, whereas Microsoft's Level 4 ("full MLOps automated operations") would require multi-environment claims the platform does not make. And it is an open, widely cited industry whitepaper [10], the most-cited maturity reference in the Zarour review, giving the assignment an external touchpoint a reviewer can check. The limitation is disclosed plainly: all five models are descriptive ordinals that support *positioning*, not *measurement*; the instruments of §5.4–§5.7 carry the measurement load.

5.9 Threats to validity (preview)

The detailed validity discussion accompanies the results in §7.6; previewed here, with one line per threat type framed per Runeson and Höst [11], are the threats the evaluation

must defend against (Table 5.9).

Table 5.9: Threats to validity at a glance; each is discussed under §7.6.

Threat	Source in this work	Mitigation
Construct	ML Test Score is self-administered	Per-item evidence in `ml_test_score.md`
Internal	Single-system observation; drift conclusion conditional	State the boundary condition; objective measurements carry main load
External	Single-case design	Analytic generalisation only; no statistical claim about MLOps platforms
Reliability	Would another evaluator score the same?	Explicit scoring rules + recorded evidence per item

5.10 What is not tested and why

Stating the evaluation's non-claims explicitly (Table 5.10) prevents the results from being over-read: each entry is something a reader might reasonably expect but which the study window or design cannot support.

Table 5.10: Explicit non-claims of this evaluation framework.

Not claimed	Why
Long-run uptime / availability	Prometheus tier is only days old; passive uptime over a short window is uninformative
Change failure rate / MTTR	No incident corpus in the window (see §5.6)
DORA tier placement	Tier thresholds re-clustered yearly; unsafe on a short window
Comparison vs another MLOps platform	Single-case design; no controlled comparison
Long-run forecasting accuracy	Belongs to the modelling-side chapter (Ch. 6), not this framework
Statistical generalisation	Single case; only analytic generalisation per [11]

6 Results — Forecasting Models

This chapter reports the head-to-head comparison of the three forecasting models specified in Chapter 3. Six backtests were run—LEAR, the DNN, and the Transformer, each on DK1 and DE—over a single shared hourly test window (2025-10-24 to 2026-05-31) with the paper-exact feature sets. Within each market the models are compared on the test days that all of them produced; because the Transformer's fourteen-day input sequence consumes more leading days than the other pipelines, this common set is 187 days on DK1 and 176 on DE. The chapter reports headline accuracy (§6.1), statistical significance (§6.2), and computation cost (§6.3), then sets the results against the published benchmarks (§6.4) and closes with the modelling-side threats to validity (§6.5) and future work (§6.6).

6.1 Headline accuracy

Table 6.1: DK1: final benchmark metrics (test 2025-10-24–2026-05-31, common days).

	MAE	RMSE	sMAPE	rMAE
LEAR	17.574	26.891	29.048	0.469
DNN	17.960	25.634	29.339	0.482
Transformer	20.076	28.909	31.767	0.543
Naive (weekly)	36.899	51.385	50.110	1.000

Table 6.2: DE: final benchmark metrics (test 2025-10-24–2026-05-31, common days).

	MAE	RMSE	sMAPE	rMAE
LEAR	13.069	25.041	22.107	0.376
DNN	13.476	22.544	22.763	0.388
Transformer	14.719	23.755	23.901	0.420
Naive (weekly)	35.049	51.914	46.090	1.000

Tables 6.1 and 6.2 report the four accuracy metrics on the common test days. The ordering is the same in both markets and across MAE, rMAE, and sMAPE: LEAR is the most accurate, the DNN follows closely, and the Transformer is third, with all three improving substantially on the weekly-naive baseline—rMAE values of 0.469 (LEAR), 0.482 (DNN), and 0.543 (Transformer) on DK1, and 0.376, 0.388, and 0.420 respectively on DE. Because rMAE normalises each model against the same-market naive error, these figures are comparable across markets: all three models sit markedly further below 1 on DE than on DK1, indicating that the German price was the more predictable of the two relative to its own naive benchmark over this window.

The single metric that reorders the models is RMSE. The DNN attains the lowest RMSE in both markets (25.6 against LEAR's 26.9 on DK1; 22.5 against 25.0 on DE), and on DE LEAR in fact posts the highest RMSE of the three, behind even the Transformer. As RMSE weights large deviations more heavily than MAE, this is evidence that the DNN suppresses the largest forecast errors more effectively than LEAR even on the days where LEAR's mean error is slightly lower. LEAR's advantage is thus specific to the absolute-error metrics that Lago et al. [3], §5.4, designate as primary; on the squared-error criterion

the DNN is preferable. Whether either difference is statistically meaningful is the subject of §6.2.

6.2 Statistical significance

Table 6.3: DK1: GW multivariate p-values (column model better than row model).

	LEAR	DNN	Transformer	Naive (weekly)
LEAR	–	0.836	1.000	1.000
DNN	0.164	–	1.000	1.000
Transformer	0.001	0.001	–	1.000
Naive (weekly)	0.000	0.000	0.000	–

Table 6.4: DE: GW multivariate p-values (column model better than row model).

	LEAR	DNN	Transformer	Naive (weekly)
LEAR	–	0.807	0.988	1.000
DNN	0.193	–	0.994	1.000
Transformer	0.012	0.006	–	1.000
Naive (weekly)	0.000	0.000	0.000	–

The Giacomini–White multivariate test [8, 3] establishes whether these differences are significant. Tables 6.3 and 6.4 give the pairwise p-values, where cell (r, c) is the probability of a loss differential at least as large as the one observed under the null that column model c is no more accurate than row model r (the convention of [3], Fig. 5); a value below 0.05 means that c significantly outperforms r.

The central result is that LEAR and the DNN are statistically indistinguishable on both markets: the hypothesis that LEAR is no more accurate than the DNN cannot be rejected (DK1 $p = 0.16$, DE $p = 0.19$), and neither can its converse. LEAR's numerical lead in the headline tables therefore does not amount to a significant accuracy advantage, and the two models should be treated as equivalent in accuracy. Both, however, significantly outperform the Transformer—on DK1 at $p = 0.001$ for each, and on DE at $p = 0.012$ (LEAR) and $p = 0.006$ (DNN)—and all three beat the weekly-naive baseline with $p \approx 0$. The comparison that survives significance testing is thus a two-way tie between LEAR and the DNN at the top, a significantly weaker Transformer, and a baseline that every model clears by a wide and significant margin, consistently across DK1 and DE.

6.3 Computation cost

Table 6.5: Amortised recalibration cost (full-backtest wall-clock / common test day). Full timings in `Report/metrics/run_timings.md`.

Model	**DK1 s/day**	**DE s/day**	**Hardware / cadence**
LEAR	50.3	41.3	CPU; one LassoCV per period $\times$ 3 windows, daily
DNN	0.9	1.6	GPU; one MIMO net (single seed), daily
Transformer	1.6	1.9	GPU; one MIMO net, weekly refit

Table 6.5 reports the amortised recalibration cost—the full-backtest wall-clock divided by the number of common test days—for each model and market. The figures invert the

cost ordering of Lago et al. [3], Table 4, where on hourly CPU hardware LEAR trains in seconds and the DNN in minutes. Here the relationship is reversed: the GPU-trained MIMO networks recalibrate in one to two seconds per day, whereas LEAR—which fits an independent cross-validated Lasso for each of the 24 delivery periods across three calibration windows, entirely on CPU—takes forty to fifty. The DNN is therefore roughly 25 to 55 times cheaper per recalibration than LEAR while, as §6.2 showed, being its statistical equal in accuracy. This inversion is a property of the hardware rather than of the models or the markets—a GPU rewards the dense neural networks and penalises LEAR's many small per-period fits—but on this platform it is the decisive practical difference between two otherwise-tied models: the DNN buys equal accuracy at around a fortieth of the compute. The Transformer, refitted weekly, carries a comparably low amortised cost but does not convert it into competitive accuracy.

6.4 Comparison with published benchmarks

A direct comparison of error levels with the source papers is complicated by the shift in price regime: the benchmarks of Lago et al. [3] were struck on 2016–2018 data, whereas this study covers 2025–2026, so only rMAE levels and rankings are comparable, never absolute errors ([3], §5.4). Table 6.6 sets the ensemble rMAE values against the paper's Table 3 figures for the corresponding markets.

Table 6.6: Our ensemble rMAE vs published ([3] Table 3). Periods differ; rankings/levels only.

	DK1 ours	**NP [3]**	**DE ours**	**EPEX-DE [3]**
LEAR	0.469	0.420	0.376	0.433
DNN	0.482	0.407	0.388	0.374

The levels are broadly in line with the published benchmark. On DK1 both models are somewhat less accurate than the paper's Nord Pool figures (0.469 against 0.420 for LEAR, 0.482 against 0.407 for the DNN), whereas on DE the LEAR ensemble is actually more accurate than the paper's EPEX-DE result (0.376 against 0.433) and the DNN is marginally above it (0.388 against 0.374). That the absolute rMAE levels land in the same range as a benchmark struck on entirely different data is a modest validation of the replication in its own right.

The rankings, however, invert both source papers. Lago et al. [3] report the DNN as at least the equal of LEAR, and Llorente and Portela [6] report their Transformer ahead of the DNN; the ordering here is the reverse on both counts, LEAR $\geq$ DNN $\geq$ Transformer. As §6.2 showed, the LEAR–DNN inversion is only numerical—the two are statistically tied, which is itself consistent with Lago et al.'s observation that the gap is often insignificant—but the Transformer's third place is genuine and significant. The likeliest cause is that the deep models are under-optimised relative to LEAR in this study rather than inherently weaker, a point developed in §6.5: the DNN was given 30 tuning trials against the paper's 1500, and the Transformer reused the paper's Nord Pool hyperparameters without per-market retuning and was adapted into the rolling-refit pipeline rather than run in its native train-once-and-renormalise mode (Chapter 3, Table 3.4), while LEAR re-selects its penalty by cross-validation at every recalibration and so sits near its achievable best.

6.5 Threats to validity (modelling side)

Several constraints bound the strength of these conclusions, all of them stemming from data availability or compute budget rather than from the model specifications. The test window is 31 weeks, short of the one-to-two years the source papers recommend, because the usable price history begins only in March 2022 and the validation and calibration windows consume the remainder. Each deep-model configuration is evaluated from a single backtest run at a fixed seed: the DNN's four-seed averaging is applied only on the production serving path and does not enter the reported backtest, and the Transformer is likewise a single seed, so both deep models carry the seed variance of one draw—a caveat that bears most heavily on the Transformer's third-place finish. The deep models are, as noted, tuned far less than LEAR (Chapter 3, Table 3.4), so the rankings should be read as contingent on the tuning budget. Finally, the 2025–2026 regime contains negative-price and high-renewable-infeed days largely absent from the papers' data, and both fifteen-minute markets are downsampled to an hourly mean for comparability; these choices affect every model identically and so do not bias the paired tests, but they do limit comparability with the original studies.

6.6 Future work (modelling side)

The most direct extension is to give the deep models a tuning budget commensurate with LEAR's effective one before any architectural conclusion is drawn: a full-scale TPE search for the DNN, approaching the paper's 1500 trials, and a per-market hyperparameter search for the Transformer in place of the reused Nord Pool defaults. Beyond tuning, three further directions follow from the deviations documented in Chapter 3: running the Transformer in its native train-once-and-renormalise mode [6] rather than the calibration-window refit used here; restoring the 1456-day LEAR window and the LARS+AIC penalty selection of the original specification once the data history is long enough to support them; and extending the point forecasts to probabilistic ones evaluated by pinball loss or CRPS. A per-hour Diebold–Mariano profile, in place of the single daily-aggregate statistic, would further localise where in the day the models diverge.

7 Results — System Evaluation

This chapter reports the system-side evaluation defined in Chapter 5; the forecast-accuracy results are in Chapter 6. Following the single-case study guidelines of Runeson and Höst [11], the system is evaluated from its source code, version-control and CI history, the ClickHouse experiment tracker, and short targeted experiments. The figures are indicative of the system over the study window rather than claims about long-run production operation — a framing that matches comparable MLOps evaluations in the electricity-forecasting domain [5]. No controlled comparison against a second platform is attempted. The instruments themselves, and the reasoning for choosing them, were set out in Chapter 5; here each is applied and its result reported.

7.1 ML Test Score

The ML Test Score rubric of Breck et al. [12] scores 28 tests across four sections on a $\{0, 0.5, 1\}$ scale, the final score being the minimum of the four section sums so that the weakest area bounds the result. Table 7.1 reports the per-section sums; the full per-item evidence is in Appendix B.

Table 7.1: ML Test Score self-assessment (min-of-sections rule). Per-item evidence in Appendix B.

Section (max 7)	**Sum**
Data tests	2.5
Model development tests	3.0
ML infrastructure tests	4.0
Monitoring tests	4.0
ML Test Score (min)	**2.5**

The final score is **2.5**, bound by the data-testing section. The system scores there for enforcing strict input schemas (`enforce_schema`) and for a tested feature-transformer pipeline, but does not formally document the feature-benefit, feature-cost, or privacy analyses the rubric also rewards, and has no data-quality CI gate. We read this as the system's primary remaining investment area (§7.7). The score is deliberately conservative: two data items — that the Lasso performs feature selection, and that the feature set is grounded in Lago et al. [3] — could each be argued to 0.5, which would lift the data section to 3.0 and the headline to 3.0; we leave them at zero absent a documented analysis, so the reported score is a lower bound.

7.2 Reproducibility

Reproducibility is reported in the two senses defined in Chapter 5: provenance coverage across all runs, and a worked re-run of one. The framing follows Gundersen and Kjensmo [13], and the badge vocabulary follows the ACM Artifact Review and Badging policy [14].

Of the 15 live runs in the window, 66.7 % carry the complete provenance triple (Table 7.2). The shortfall is entirely on `git_sha`: the data-window and artefact columns are populated for every run from day one, while five early runs predate the CD pipeline that bakes

Table 7.2: Provenance triple coverage across 15 live runs in the window.

Provenance component	Coverage
`git_sha`	66.7 % (10 / 15)
`data_start_utc`	100.0 % (15 / 15)
`data_end_utc`	100.0 % (15 / 15)
`model_artifact_id`	100.0 % (15 / 15)
Full triple (all four)	**66.7 % (10 / 15)**

`GIT_SHA` into the production image. From the first cd-deployed run onward, all four components are present.

Table 7.3: Reproducibility re-run: regenerated from logged provenance, compared against the stored prediction.

Field	Value
Config	`dk1_lear`
Target date	2026-03-22
Delivery periods	24
Max absolute difference	7×10^{-6} EUR/MWh
Verdict	numerically identical
Command	`uv run python -m scripts.report.reproduce_run --config dk1_lear --date 2026-03-22`

For the demonstration (Table 7.3), the only inputs supplied were the run's committed config and target date; the stored prediction was loaded from `algo.experiment_predictions`. For the deterministic LEAR configuration on 2026-03-22, the maximum absolute difference across all 24 delivery periods was 7×10^{-6} EUR/MWh — five orders of magnitude below any economically meaningful price unit, i.e. numerically identical up to solver-level floating-point tolerance. LEAR is used for the headline demonstration because it is deterministic given a fixed data window; the DNN and Transformer carry seed and ensemble stochasticity whose controlled re-run is left to future work. Against the ACM vocabulary [14], the artefact is Available, Functional, and Reusable by self-review; the independent "Reproduced" badge, which requires an external reviewer, is not claimed.

7.3 Delivery throughput

Treating each successful run of `cd_deploy.yml` on `main` as one deployment, the two DORA throughput metrics [15] over the window are 20 successful deployments (1.56 per week), with a PR-merge-to-deploy-start lead time of 0.1 min at both the median and the 95th percentile. Change failure rate and mean time to restore are deliberately not reported — both require an incident corpus that a study window of this length does not provide — and no DORA performance tier is assigned, since the tier thresholds are re-derived annually. The sub-minute lead time reflects that `cd_deploy.yml` fires immediately on push to `main` and that the interval measured runs from the merge commit to the workflow's own start, not from the first commit on a feature branch.

7.4 Drift detection as monitoring, not a retraining trigger

The system refits every model from scratch daily on a rolling window at a cost of seconds (Chapter 4, P6), so the model is never stale between scheduled refits. In this regime, drift-*triggered* retraining adds no freshness benefit over the scheduled refit: the cost–benefit analysis of Mahadevan and Mathioudakis [16] shows that triggered retraining earns its keep only when retraining is expensive, and that distribution-drift triggers otherwise incur unnecessary retrains from false positives. Drift detection is therefore retained as a *monitoring and alerting* signal rather than a retraining trigger, which is why `retrain_on_drift` defaults to false. This conclusion is conditional: it holds while retraining stays cheap and the rolling window short, and would be revisited for expensive multi-year Transformer retraining. One calibration limitation is noted: over the window the detector flagged drift on 49 of 49 reports, a detection rate that is uninformatively high and is listed as future work in §7.7.

7.5 Maturity placement

The platform is positioned against the Google three-level maturity model, the choice of which is justified in Chapter 5, §5.8. Table 7.4 assigns a status and a Google level to each of Zarour's nine practices, with one line of evidence apiece. The three status labels map to the evidence already presented: a practice is *Realised* when a concrete, production-running artefact closes it end-to-end, and *Minimal* when the mechanism exists but its scope is intentionally restricted. Eight of the nine are Realised; the ninth, the feedback loop, is Minimal — drift-triggered retraining exists but is scoped to two areas behind an opt-in flag, consistent with the monitoring-not-trigger argument of §7.4. The levels themselves are descriptive ordinals and support positioning, not measurement; the quantitative load is carried by the ML Test Score and reproducibility results above.

7.6 Threats to validity

The threats are framed per Runeson and Höst [11]. **Construct validity** — do the chosen scores measure what is claimed? — is the main concern for the ML Test Score, which is a self-administered rubric; the per-item evidence in Appendix B makes every score checkable and contestable. **Internal validity** — are the observed properties causally attributable to the platform? — is strongest for the reproducibility results, which speak directly from committed artefacts rather than inference. **External validity** is bounded by the single-case design: the findings support analytic generalisation to platforms sharing this one's characteristics, not statistical claims about MLOps platforms in general. **Reliability** — would another evaluator reach the same scores? — is supported by the explicit scoring rule and the recorded per-item evidence. These threat types parallel the validity discussion in Zarour et al.'s own review [4].

7.7 Known gaps and future work

Three system-side gaps follow from the results. The binding **data-testing** section would be raised by formalising the feature-benefit, feature-cost, and privacy analyses and adding a data-quality CI gate. The **reproducibility** demonstration currently covers only the deterministic LEAR; extending it to the DNN and Transformer requires a controlled-seed re-run that suppresses GPU non-determinism. The **provenance** shortfall is confined to five pre-CD runs lacking `git_sha`, which could be backfilled from log timestamps or left documented as a one-off. The **drift detector** flagged every report as drifted over the window, an uninformative rate that calls for threshold recalibration. Modelling-side future work is collected in Chapter 6.

7.8 Summary

The chapter turns the qualitative “Realised” labels of Table 7.4 into measured values: an ML Test Score of 2.5 (bound by data testing), provenance-triple coverage of 66.7 % backed by a re-run that reproduced a logged forecast to within 7×10^{-6} EUR/MWh, and delivery-throughput figures of 20 deployments at a sub-minute lead time. On the Google model the platform sits at Level 2 for the CI/CD, orchestration, reproducibility, versioning, and monitoring practices, and Level 1 for collaboration, continuous training, metadata tracking, and the feedback loop. Each objective figure is recomputable on demand by re-running `scripts/report/extract_metrics.py` and `scripts/report/reproduce_run.py` against the live system.

Table 7.4: Maturity assessment of Zarour's nine practices, with the Google MLOps maturity model level [10] assigned to each.

Practice	Status	Google level	One-line evidence
P1 — Automating CI/CD	Realised	2	`pr_test`/`pr_ruff-lint`/`pr_smoke` PR gates; `cd_deploy` on every merge to `main`
P2 — Orchestrating workflows	Realised	2	Prefect flows with per-area deployments on a Docker work pool
P3 — Ensuring reproducibility	Realised	2	`GIT_SHA` + data-window columns per run; containerised runtime in `docker-compose.yml`
P4 — Versioning	Realised	2	Git: `configs/`, numbered migrations, dashboards and alert rules all version-controlled
P5 — Collaboration & communication	Realised	1	`CLAUDE.md`, `REPO_OVERVIEW.md`, `RUNBOOK.md`, `CONTRIBUTING.md`, plus Slack alerts
P6 — Continuously train & evaluate	Realised	1	Six `daily-forecast` deployments refit at 06:00 UTC; reconciliation scores live forecasts
P7 — Track & log ML metadata	Realised	1	`algo.experiment_runs` + predictions + artifacts, plus the W&B research surface
P8 — Continuous monitoring	Realised	2	Loki plus three Grafana dashboards (Data Quality, Operational, System Health) and four file-based alert rules; model metrics in W&B `algo-prod`
P9 — Implementing feedback loops	Minimal	1	Drift-triggered retrain behind a config flag, two areas only

8 Discussion

This chapter answers the three research questions of §1 and then draws the forecasting and system threads together. The forecasting answer (RQ1) rests on results reported in Chapter 6; the engineering and maturity answers (RQ2, RQ3) synthesise the system implementation and its evaluation.

8.1 RQ1 — Comparing the forecasting models

The head-to-head comparison of LEAR, the DNN, and the Transformer across DK1 and DE is reported in Chapter 6, on a single shared test window with paired Diebold–Mariano and Giacomini–White tests, and against both a weekly-naive baseline and the rankings of Lago et al. [3] and Llorente and Portela [6]. Because the source papers' test periods predate the present price regime, the comparison is read through rMAE and model rankings rather than absolute errors.

Across both bidding zones the three models fall into two tiers. LEAR records the lowest mean absolute error—and with it the lowest rMAE and sMAPE—on both DK1 and DE (rMAE 0.469 and 0.376), the DNN follows closely (0.482 and 0.388), and the Transformer is third (0.543 and 0.420); all three improve substantially on the weekly-naive baseline. The Giacomini–White test, however, cannot distinguish LEAR from the DNN on either market: the hypothesis that LEAR is no more accurate than the DNN gives $p = 0.16$ on DK1 and $p = 0.19$ on DE, and neither can its converse be rejected. LEAR's numerical lead is therefore not a significant accuracy advantage, and the two models are best regarded as equivalent. The same test confirms that LEAR and the DNN each significantly outperform the Transformer ($p \leq 0.012$) and that every model beats the naive forecast ($p \approx 0$). The answer to RQ1 is thus a statistical tie between LEAR and the DNN at the top, a significantly weaker Transformer, and a wide, significant margin over the naive benchmark—a pattern that holds identically on DK1 and DE.

Two refinements qualify this headline. First, the ranking depends on the error metric: although LEAR leads on the absolute-error measures, the DNN achieves the lowest RMSE in both markets, and on DE LEAR posts the highest RMSE of the three models. Because RMSE penalises large errors more heavily than MAE, the DNN evidently controls the worst misforecasts better than LEAR even where its mean error is marginally larger—a distinction that matters whenever large individual errors are disproportionately costly. Second, the accuracy tie does not extend to cost: on the platform's GPU the DNN recalibrates in about a second per day against LEAR's forty to fifty (Table 6.5), so it delivers statistically equal accuracy at a fraction of the compute. Where two models are tied on accuracy, this asymmetry becomes the deciding practical factor, and it favours the DNN.

Measured against the source papers, the comparison replicates one result and inverts two. It reproduces the finding of Lago et al. [3] that the accuracy gap between LEAR and the DNN is frequently insignificant—here the two are statistically tied. It does not reproduce the papers' point rankings: Lago et al. place the DNN at least level with LEAR, yet here LEAR is numerically ahead (though tied), and Llorente and Portela [6] place their Transformer above the DNN, yet here the Transformer is significantly the weakest model. The most credible explanation is procedural rather than architectural. The deep models were optimised far less than LEAR in this study: the DNN was tuned for 30 TPE trials against the paper's 1500, and the Transformer reused the paper's Nord Pool hyperparam-

eters without per-market retuning, adapted into the rolling calibration-window-and-weekly-refit pipeline of the other models rather than its native train-once-and-renormalise scheme (Chapter 3, Table 3.4). LEAR, by contrast, re-selects its penalty by cross-validation at every daily recalibration and so sits close to its achievable best. The inversion is therefore better read as a measure of how much tuning the deep models need to express their reported advantage than as evidence that their architectures are weaker; narrowing that gap is the leading item of modelling future work (Chapter 6, §6.6).

8.2 RQ2 — Engineering trade-offs of productionising an EPF benchmark

Embedding the benchmark models in a scheduled, containerised pipeline surfaced a series of trade-offs, each resolved in favour of the single-host MVP scope. The most consequential is the choice of a full refit from scratch on every daily run over warm-start or incremental learning: because the daily data increment is small relative to the calibration windows, a clean refit is cheap and avoids accumulated approximation error, and — as RQ3 and the drift discussion show — it also makes the model fresh enough that drift-triggered retraining becomes redundant. Co-locating training and prediction in a single Prefect flow trades a small amount of recomputation for the guarantee that the served model always matches the forecast it produced, removing a class of coordination failure. Using ClickHouse as the single store for time-series, predictions, metrics, and model artifacts makes day-after reconciliation one query rather than a cross-system join, at the cost of a base64-blob artifact representation that will not scale to large neural weights. And serving the paper-strict feature set in production — rather than a separate, richer operational feature set — keeps the deployed models bit-faithful to the benchmark, at the cost of forgoing features that might help operationally but would break the replication. The recurring pattern is that the constraints of a faithful benchmark and the constraints of a small production platform mostly align: both reward simplicity, determinism, and a single source of truth.

8.3 RQ3 — Critical practices and the limits of a single-host platform

Of Zarour's nine practices, eight are realised end-to-end and one — the feedback loop — is minimally realised (Table 7.4). The quantitative evaluation locates the binding constraint precisely: the ML Test Score of 2.5 is bound by the data-testing section, not by infrastructure or monitoring, which both score higher. In other words, for this platform the practices that proved easiest to realise to a high level were the DevOps-inherited ones — CI/CD automation, versioning, containerised reproducibility, and monitoring — because they transfer directly from software engineering, whereas the ML-specific data-quality discipline (feature-benefit, feature-cost, and slice analyses) is where the platform is weakest and where further investment would most raise its maturity. The single-host scope reaches its limit at three identifiable points: the artifact store, which suits LEAR's hundred-kilobyte blobs but not multi-megabyte neural weights; the absence of a model-quality gate before serving; and the coverage gap, with forecasting and drift scoped to two of the seven ingested bidding zones. None of these is a flaw in the design so much as a deliberate boundary of the MVP, each with a named adoption trigger (Chapter 4).

8.4 Tying the threads together

The two threads are mutually reinforcing rather than independent. Several modelling decisions are only defensible because of the system around them: downsampling both

markets to an hourly grid is what makes the cross-market and cross-paper comparison valid; and the cheap, scheduled daily refit is what licenses treating drift detection as monitoring rather than as a retraining trigger, following the cost-aware argument of Mahadevan and Mathioudakis [16]. Conversely, the system's central claim — that any forecast can be reconstructed from its logged provenance — is what makes the forecasting replication itself trustworthy: the worked re-run that reproduced a logged LEAR forecast to within 7×10^{-6} EUR/MWh (§7.2) is evidence about the modelling thread as much as the system thread. The forecasting payload exercises and validates the platform, and the platform makes the forecasting reproducible; neither half of the thesis would stand on its own.

8.5 Limitations

The limitations span both threads and are collected here. On the modelling side, the test window is 31 weeks rather than the one-to-two years the source papers recommend, a consequence of the data history; the DNN hyperparameters are tuned on an architecture-only search space, with Lago's ten binary feature-selector hyperparameters left as fixed config decisions because the preprocessor exposes no toggles for them; the Transformer reuses the paper's search-range defaults rather than per-market optima; and the work forecasts point prices, not probabilistic distributions. On the system side, the evaluation is a single-case study with no controlled comparison, the ML Test Score is self-administered (its per-item evidence in Appendix B is the mitigation), forecasting and drift cover only DK1 and DE, the ClickHouse blob artifact store will not scale to large neural weights, and the drift detector over-flags. These are revisited as concrete future work in Chapter 9, and the modelling-specific items are detailed alongside the results in Chapter 6.

9 Conclusion and Future Work

This thesis took the benchmark electricity-price-forecasting models from the research literature and embedded them in an end-to-end production platform built along MLOps principles, then evaluated the result on two axes: the forecast quality of the models and the operational maturity of the system serving them.

On the system axis, eight of Zarour et al.'s nine MLOps practices [4] are realised end-to-end in the platform, and one — the feedback loop — is minimally realised, scoped to two bidding zones behind an opt-in flag. Every realised practice is traced to a concrete repository artefact: a Prefect deployment, a ClickHouse migration, a GitHub Actions workflow, or a Grafana dashboard. The qualitative "realised" labels are backed by measured values — an ML Test Score of 2.5 bound by data testing, provenance-triple coverage of 66.7 % supported by a re-run that reproduced a logged forecast to within 7×10^{-6} EUR/MWh, and DORA throughput of 20 deployments at a sub-minute lead time — each recomputable on demand from the live system. On the forecasting axis, the faithful replication of LEAR, the DNN, and the Transformer across DK1 and DE, evaluated with paired significance tests on a shared recent window, is reported in Chapter 6: on both markets LEAR proved the most accurate model and the DNN its statistical equal—indistinguishable under the Giacomini–White test ($p > 0.16$)—while both significantly outperformed the pure Transformer and all three beat the weekly-naive baseline. This ordering inverts the rankings of the source papers, in which the DNN edges LEAR [3] and the Transformer leads the DNN [6]; the inversion is attributable to the deep models' reduced tuning budget in this study rather than to any inherent weakness of their architectures. Together, the two threads constitute a production ML platform that ingests live market data, recalibrates and forecasts daily, monitors prediction quality and data drift, and maintains a complete audit trail from every forecast back to the exact code commit and data window that produced it.

9.1 Future work

Several extensions follow directly from the limitations of §7.7 and Chapter 8.

- **Modelling fidelity.** Restore the 1456-day calibration window and the LARS+AIC λ-selection of the original benchmark once the data history is long enough; tune the Transformer per market rather than reusing the paper defaults; and run an `epftoolbox` side-by-side on the original 2016–2018 datasets to separate replication error from regime change.
- **Calibration-window sensitivity.** Use the `forecast-cli ablate` command to quantify how accuracy varies with calibration-window length, an ablation the present fixed-window matrix does not cover.
- **Probabilistic forecasting.** Extend the point forecasts to predictive distributions and evaluate with pinball loss or CRPS, the direction surveyed by Nowotarski and Weron [2].
- **Reproducibility of stochastic models.** Demonstrate the re-run for the DNN and Transformer under controlled seeds with GPU non-determinism suppressed, extending the deterministic-LEAR demonstration.
- **Data-testing maturity.** Formalise the feature-benefit, feature-cost, and privacy analyses and add a data-quality CI gate — the binding constraint on the ML Test

Score.

- **Drift recalibration and coverage.** Recalibrate the drift detector, which currently flags every report, and extend forecasting and drift to the remaining ingested bidding zones.
- **Tooling adoption triggers.** Adopt MLflow, Kubernetes, a cloud-managed model store, or DVC if and when their named, observable triggers are reached (Chapter 4).

Bibliography

[1] Rafał Weron. “Electricity price forecasting: A review of the state-of-the-art with a look into the future”. In: *International Journal of Forecasting* 30.4 (2014), pp. 1030–1081. DOI: 10.1016/j.ijforecast.2014.08.008.

[2] Jakub Nowotarski and Rafał Weron. “Recent advances in electricity price forecasting: A review of probabilistic forecasting”. In: *Renewable and Sustainable Energy Reviews* 81 (2018), pp. 1548–1568. DOI: 10.1016/j.rser.2017.05.234.

[3] Jesus Lago et al. “Forecasting day-ahead electricity prices: A review of state-of-the-art algorithms, best practices and an open-access benchmark”. In: *Applied Energy* 293 (2021), p. 116983. DOI: 10.1016/j.apenergy.2021.116983.

[4] Mohammad Zarour, Hussein Alzabut, and Khalid T. Al-Sarayreh. “MLOps best practices, challenges and maturity models: A systematic literature review”. In: *Information and Software Technology* 183 (2025), p. 107733. DOI: 10.1016/j.infsof.2025.107733.

[5] Rakshith Subramanya, Seppo Sierla, and Valeriy Vyatkin. “From DevOps to MLOps: Overview and Application to Electricity Market Forecasting”. In: *Applied Sciences* 12.19 (2022), p. 9851. DOI: 10.3390/app12199851.

[6] Jesús Llorente and José Portela. “A Transformer Approach for Electricity Price Forecasting”. In: *arXiv preprint arXiv:2403.16108* (2025).

[7] Francis X. Diebold and Roberto S. Mariano. “Comparing Predictive Accuracy”. In: *Journal of Business & Economic Statistics* 13.3 (1995), pp. 253–263.

[8] Raffaella Giacomini and Halbert White. “Tests of Conditional Predictive Ability”. In: *Econometrica* 74.6 (2006), pp. 1545–1578.

[9] Bartosz Uniejewski, Rafał Weron, and Florian Ziel. “Variance Stabilizing Transformations for Electricity Spot Price Forecasting”. In: *IEEE Transactions on Power Systems* 33.2 (2018), pp. 2219–2229. DOI: 10.1109/TPWRS.2017.2734563.

[10] Google Cloud. *MLOps: Continuous delivery and automation pipelines in machine learning*. Google Cloud Architecture Center whitepaper. Defines a three-level MLOps maturity model (Level 0 manual, Level 1 ML pipeline automation, Level 2 CI/CD pipeline automation). 2024. URL: https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning.

[11] Per Runeson and Martin Höst. “Guidelines for conducting and reporting case study research in software engineering”. In: *Empirical Software Engineering* 14.2 (2009), pp. 131–164. DOI: 10.1007/s10664-008-9102-8.

[12] Eric Breck et al. “The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction”. In: *2017 IEEE International Conference on Big Data (Big Data)*. 2017, pp. 1123–1132. DOI: 10.1109/BigData.2017.8258038.

[13] Odd Erik Gundersen and Sigbjørn Kjensmo. “State of the Art: Reproducibility in Artificial Intelligence”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1. 2018. DOI: 10.1609/aaai.v32i1.11503.

[14] Association for Computing Machinery. *Artifact Review and Badging — Version 1.1*. ACM editorial policy. Defines the Artifacts Available; Artifacts Evaluated (Functional / Reusable); and Results Reproduced / Replicated badges. 2020. URL: https://www.acm.org/publications/policies/artifact-review-and-badging-current.

[15] Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

[16] Ananth Mahadevan and Michael Mathioudakis. “Cost-aware retraining for machine learning”. In: *Knowledge-Based Systems* 293 (2024), p. 111610. DOI: 10.1016/j.knosys.2024.111610.

A Glossary

LEAR Lasso Estimated AutoRegressive ensemble for day-ahead electricity price forecasting, the primary baseline model from Lago et al. [3].

DNN Deep neural network; in this chapter, a multi-layer perceptron ensemble whose final prediction is the arithmetic mean of `n_dnn_ensemble` replicates trained with distinct random seeds.

EPF Electricity price forecasting: the task of predicting day-ahead wholesale electricity prices for a given bidding zone from historical prices and exogenous signals.

ENTSO-E European Network of Transmission System Operators for Electricity; the body that operates the Transparency Platform API from which the platform ingests price, load, and generation data.

Lasso Least Absolute Shrinkage and Selection Operator, an L1-regularised linear regression that simultaneously shrinks coefficients and performs automatic feature selection.

TPE Tree-structured Parzen Estimator, the Bayesian optimisation sampler used by Optuna when running `forecast-cli tune`.

MLOps Machine Learning Operations: the set of practices for reliably building, deploying, and operating machine learning systems in production.

ReplacingMergeTree A ClickHouse storage engine that deduplicates rows sharing the same sort key during background merge, used throughout the platform to make re-insertion safe under the platform's append-only write semantics.

Tailscale A mesh VPN built on WireGuard used here to grant the GitHub Actions runner ephemeral SSH access to the deploy host without exposing any port on that host to the public internet.

Loki Grafana Labs' log aggregation system, used here as the structured-log backend for all Prefect flows and infrastructure modules via direct Python push through `python-logging-loki`.

Evidently An open-source library for data and model drift detection, used here in in-process mode inside the Prefect drift-detection flow to apply its `DataDriftPreset`, which auto-selects a drift test (Kolmogorov–Smirnov or Wasserstein, by sample size) and compares a 90-day reference window against a 7-day current window.

B ML Test Score Worksheet

The full per-item ML Test Score worksheet of Breck et al. [12] follows. Each item is scored 0 / 0.5 / 1 by the rule in Table 5.4, with one line of evidence pointing into the repository. The section sums are Data 2.5, Model development 3.0, ML infrastructure 4.0, and Monitoring 4.0; the final score is their minimum, **2.5** (§7.1).

Table B.1: Section 1 — Data tests (sum 2.5 / 7).

#	Test	Score	Evidence
1	Feature expectations in a schema	1	`enforce_schema()` in `domain/curves.py`; raises on dtype/column mismatch; tested in `tests/domain/`
2	All features are beneficial	0	No systematic ablation documenting each feature's contribution (`forecast-cli ablate` exists but was not used for this)
3	No feature costs too much	0	No per-transformer latency/compute-cost analysis; no documented cost threshold
4	Features meet meta-level requirements	0	No regulatory/contractual/fairness requirements apply to wholesale price data
5	Pipeline has privacy controls	0	ENTSO-E data is public market data with no PII; n/a domain, scored 0
6	New features added quickly	0.5	`FeatureTransformer` ABC documented in `CONTRIBUTING.md`; manual process, no scaffold/SLA
7	All input feature code is tested	1	`tests/.../test_transformers.py` covers every transformer; runs in CI

Table B.2: Section 2 — Model development tests (sum 3.0 / 7).

#	Test	Score	Evidence
1	Model specs reviewed and submitted	0.5	`configs/*.py` are first-class Git files under PR review; no automated spec-schema gate
2	Offline and online metrics correlate	0.5	Reconciliation computes the same MAE/RMSE/sMAPE on live predictions as backtesting; no formal correlation study
3	All hyperparameters tuned	0.5	DNN: 6 HPs via Optuna; LEAR λ cross-validated; windows from the paper; Transformer partly manual
4	Impact of model staleness known	0.5	Daily refit bounds staleness < 24 h; no documented degradation experiment at 48 h / 7 d
5	A simpler model is not better	0.5	LEAR vs DNN vs Transformer tracked; rMAE vs weekly-naive; not an automated gate
6	Model quality sufficient on slices	0.5	Per-area backtests; `compute_metrics_per_hour`; no automated per-slice gate
7	Model tested for inclusion (fairness)	0	n/a domain — no demographic groups; scored 0

Table B.3: Section 3 — ML infrastructure tests (sum 4.0 / 7).

#	Test	Score	Evidence
1	Training is reproducible	0.5	`experiment_runs` carries `git_sha` + data windows + `model_artifact_id`; LEAR deterministic; GPU non-determinism not suppressed
2	Model specs are unit tested	1	`test_models.py`, `test_transformer_model.py`, `test_dnn_ensemble.py` in CI
3	Full pipeline integration tested	1	LEAR smoke test plus full Compose-stack smoke in `pr_smoke.yml`
4	Quality validated before serving	0	No quality gate before predictions are written to `experiment_predictions`
5	Debuggable by observing steps	0.5	Loki structured logging; W&B per-epoch loss; no per-layer step-through
6	Tested via a canary process	0.5	`cd_deploy smoke_run` triggers a forecast post-deploy; checks completion, not quality
7	Quick and safe rollback	0.5	Artifact blobs in `model_artifacts`; image rollback via build + restart; no automated rollback command

Table B.4: Section 4 — Monitoring tests (sum 4.0 / 7).

#	Test	Score	Evidence
1	Dependency changes notified	0	`uv.lock` tracked in Git; no Dependabot/Renovate notification
2	Data invariants hold train & serve	0.5	`enforce_schema` at construction; shared preprocessor; not independently monitored/alerted
3	Train/serve features compute alike	1	A single `DataPreprocessor` used by both `BacktestRunner` and `LiveRunner` — a structural guarantee
4	Models are not too stale	0.5	Daily refit < 24 h; Grafana `data-staleness` alert on inputs; no dedicated model-age alert
5	The model is numerically stable	0.5	`asinh` scaler; Transformer gradient clipping; no automated NaN/inf assertion on outputs
6	Compute has not regressed	0.5	`training_duration_seconds` tracked per run and in W&B; no automated threshold alert
7	Prediction quality not regressed	1	`_maybe_alert_accuracy_degradation` (MAE $> 1.5\times$ 30-day median) and the `reconciliation-missed` rule, both automated

A reproducible, monitored electricity-price-forecasting platform, together with a faithful, significance-tested comparison of the benchmark LEAR, DNN, and Transformer models on current ENTSO-E market data.

Technical
University of
Denmark

[Building / street]
2800 Kgs. Lyngby
Tlf. 4525 1700

www.dtu.dk